

Deep Learning for Explainable Cancer Identification Spectroscopy

Thomas Hartigan

2025

Abstract

In this work, I have explored the use of machine learning and explainability methods in the context of colon-cancer tissue health assessment using fourier transform infrared spectroscopy over the spectral range 1000-1800 cm^{-1} . For this, I implemented mean squared error (MSE), cosine similarity, principal component analysis (PCA) informed linear discriminant analysis (LDA), PCA informed extreme gradient boosting (XGB), standard XGB and ensemble scale-invariant convolutional neural network (SCNN) methods. Each of these were used for the classification of colon cancer samples as non-cancerous, hyperplastic, adenomatous, or cancerous. The SCNN method was found to be the most effective, achieving a weighted overall accuracy of $58.7 \pm 3.7\%$ when performing four-class classification on a four-fold cross-validated test set. When mapped to binary classification, the SCNN also performed best and performed non-cancer vs cancer classification with an overall weighted accuracy of $83.2 \pm 5.9\%$. The explainability of this model was then explored using the context-representative interpretable model explanation (CRIME) framework. This was capable of extracting multiple contexts to explain each tissue classification. The spectra and weightings associated with these contexts were then used to create a weighted cosine similarity classification model (LCRIME). The best of these LCRIME models for four-class classification utilised a 20% supervised convolutional encoder - transformer decoder variational autoencoder (VAE). This model was capable of reproducing SCNN classification predictions with an overall weighted accuracy of $47.5 \pm 7.6\%$. For binary classification, the best LCRIME model utilised a 20% supervised transformer encoder-transformer decoder VAE and achieved an overall weighted accuracy of $74.7 \pm 10.9\%$ on the same test set.

Contents

1 Introduction

Cancer is one of the leading causes of death worldwide with around 20 million cases reported in 2022. This corresponds to one in five men or women developing cancer in their lifetime [bray'global'2024]. The conventional method for cancer diagnosis, as outlined in Figure ??, involves taking a biopsy of suspect tissue from a patient during surgery, fixing this biopsy to a slide, and then staining the biopsy with haematoxylin and eosin (H&E). The haematoxylin stains nucleic acids a deep blue-purple colour, whilst the eosin is pink and non-specifically stains proteins [fischer'hematoxylin'2008]. These H&E stained samples can then be analysed by a pathologist to develop a diagnosis. This process is manual, error-prone, subjective, costly, and time-consuming; often requiring transport to a laboratory for processing by highly trained technicians [hollon'near'2020]. The sample processing time alone is often 20 minutes or more [novis'interinstitutional'1997], so the development of methods which forgo traditional lab-based preparation have significant potential to guide surgeries more effectively.

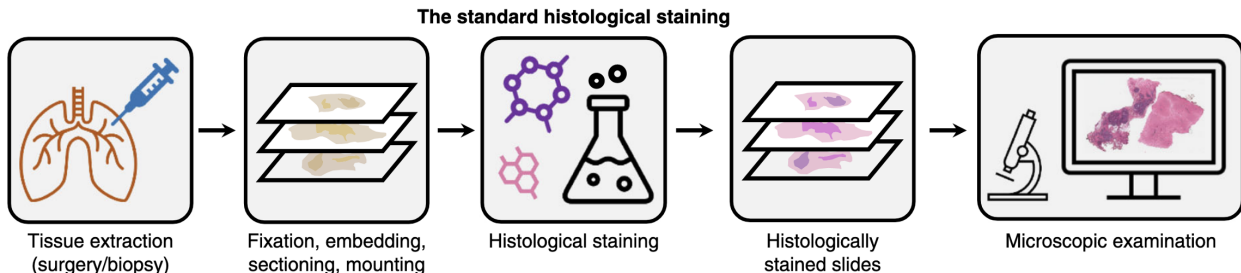


Figure 1: The standard histopathological diagnosis process. From [bai'deep'2023].

New methods are being developed to exploit machine learning (ML) in conjunction with a range of imaging technologies to expedite cancer diagnosis. The imaging technologies under investigation include stimulated Raman spectroscopy (SRS) [hollon'near'2020, sarri'fast'2019, kondepudi'foundation'2024, jiang'opensrh'2022], second harmonic generation microscopy (SHG) [sarri'fast'2019], and Fourier transform infrared spectroscopy (FTIR) [tomas'detection'2022, berisha'deep'2019, tang'advancing'2024, sala'liquid'2022, duckworth'frontline'2023]. Within this work, I focus on the application of FTIR spectroscopy to cancer identification and classification within colon cancer samples provided by collaborators at the University of Exeter [nallala'high-resolution'2016, noauthor'minerva'nodate].

1.1 Previous Works

Previous works using vibrational spectroscopy methods have reported high classification performance for cancer identification. For example, Lasalvia et al. [lasalvia'discrimination'2023] report cancer vs non-cancer classification accuracies from 87% to almost 100% for colorectal cancer samples using ML techniques on FTIR spectra. Furthermore, work by Tiwari et al. [tiwari'colon'2022] has achieved 97% and 80% colon tumour classification accuracy for high/moderate and low-grade tumours respectively using a pre-trained ResNet and convolutional network approach. Tomas et al. [tomas'detection'2022] have used attenuated total reflection FTIR (ATR-FTIR) spectroscopy and feed-forward full-connected neural networks to classify breast cancer samples into benign and malignant classifications with $96.06\% \pm 7.07\%$ accuracy. They found the different tissue types within breast cancer exhibit large differences in spectral characteristics as can be seen in Figure ?. Some works [he'accurate'2022] have also explored the use of variational autoencoders (VAEs) in classification tasks. However, their applicability has been limited mostly to de-noising input Raman spectral data. These advances demonstrate the potential for vibrational spectroscopy and machine learning in the field of cancer detection, which I aim to build on in this work.

1.2 Novelty of this Work

In this work, I present three main contributions. Firstly, weakly-labelled data is used to train a scale-invariant convolutional neural network (SCNN) for the classification of colon cancer samples into four classes: non-cancer, hyperplastic, adenoma and cancerous. Secondly, I apply explainability methods on this model to

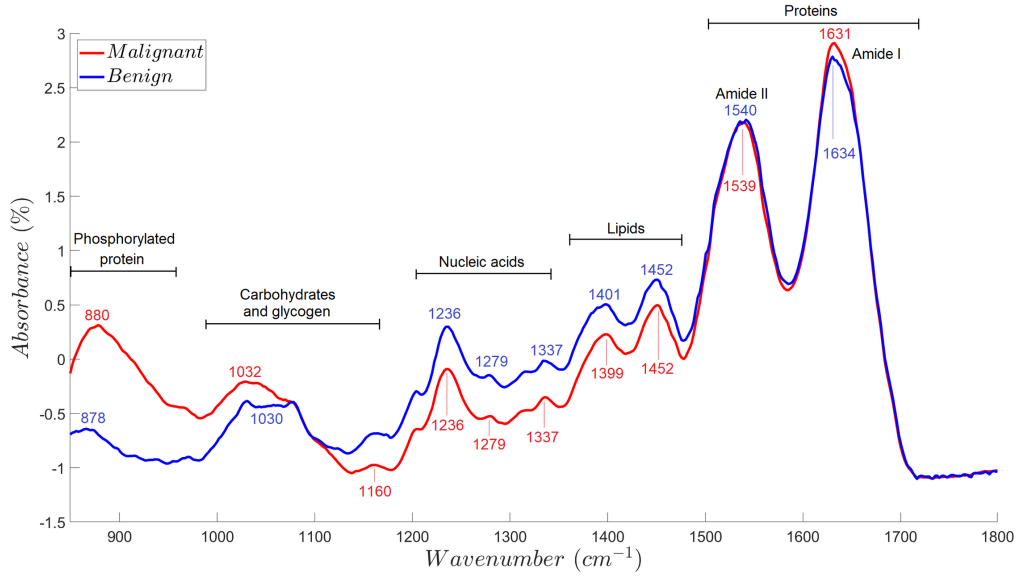


Figure 2: The IR spectral differences identified between malignant and benign tumours found in [tomas'detection'2022].

elucidate the multiple pathways by which these classifications can be achieved. Finally, these explainability results are used to create a simple weighted cosine similarity model for classification into the same four classes. The SCNN and explainability methods are inspired by contributions by Zaki et al. [zaki'explainable'2024], who used a similar approach to identify serotonin-dominated VAE latent space regions from the Raman spectra of urine samples.

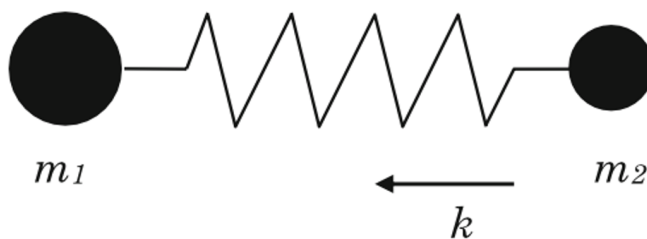
2 Theory and Methods

2.1 FTIR Spectroscopy

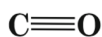
Infrared (IR) spectroscopy works by directing a broadband spectrum with wavenumbers ranging from around $400 - 4000 \text{ cm}^{-1}$ (for mid-IR spectroscopy) [su'fourier'2020] onto a sample. This coherent spectrum then excites resonant vibrational modes within molecular bonds inside the sample, causing absorption of specific frequencies depending on the composition of the incident sample. This decreases the intensity at the absorbed frequencies relative to the broadband baseline, generating a characteristic spectrum for each vibrational mode. The frequencies at which these vibrations occur are ultimately determined by quantum electron density effects [atkins'atkins'2022]. However, they are often well modelled by a simple harmonic oscillator approach [lin'calculating'2008, berthomieu'fourier'2009] subject to the requirement that the molecular dipole moment μ can change [gong'application'2024, veit'predicting'2020]. For the bond demonstrated in Figure ??, we therefore expect IR absorption at frequency:

$$\nu = \frac{1}{2\pi c} \sqrt{\frac{k(m_1 + m_2)}{m_1 m_2}}. \quad (1)$$

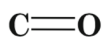
The requirement for molecules to be polar results in FTIR being particularly sensitive to functional groups with high electronegativity differences like those depicted in Figure ??.



Effect of chemical bonds :



$\sim 2100 \text{ cm}^{-1}$



$\sim 1680\text{-}1630 \text{ cm}^{-1}$



$\sim 1200 \text{ cm}^{-1}$

Effect of H-bonding :

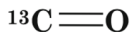


$\sim 1680\text{-}1630 \text{ cm}^{-1}$

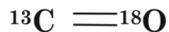


$\Delta\nu \sim - (10\text{-}20) \text{ cm}^{-1}$

Effect of mass :



$\Delta\nu \sim - (35\text{-}55) \text{ cm}^{-1}$



$\Delta\nu \sim - 65 \text{ cm}^{-1}$

Figure 3: An illustration of a bond with spring constant k between two atoms with masses m_1 and m_2 , along with demonstrations of how different levels of bonding and isotopic effects can alter IR absorption frequencies. From [berthomieu'fourier'2009].

Modern devices use a Fourier transform-based (FTIR) interferogram approach [griffiths'fourier'1983], enabling high signal-to-noise (SNR) ratios to be achieved. FTIR can detect biochemical features includ-

ing nucleic acids, proteins, lipids, and carbohydrates within biological samples by matching the observed spectra with known forms for different bonding types, functional groups and intermolecular interactions [su'fourier'2020]. Typical absorption groups within biological samples are detailed in Table ?? . Furthermore, we expect FTIR to be a good for cancer detection, as cancerous growth is often associated with changes in the acidity of tissues, and changes in acidity are known to cause hydrogen-bonding induced changes to FTIR spectral peak positions [wolpert'infrared'2006, torii'modeling'2024, czowski'cancer'2020]. This effect is demonstrated in Figure ?? .

Spectral Position (cm^{-1})	Assignment
1020	glycogen
1087	symmetric PO_2^- stretching of nucleic acids
1167	C-OH stretching of proteins
1236	asymmetric PO_2^- stretching of nucleic acids
1310	amide III of proteins
1395	COO^- stretching of proteins and lipids
1455	CH_3 bending of proteins and lipids
1542	amide II of proteins
1645	amide I of proteins
1740	C=O stretching of lipids

Table 1: Relevant FTIR spectral assignments within the wavenumber range investigated. From [lasalvia'discrimination'2023, talari'advances'2017].

2.1.1 FTIR Dataset Collection

The dataset used in this work contains FTIR spectral images of 90 colon cancer tissue samples originally collected for the MINERVA project [noauthor'minerva'nodate]. These images were produced by collaborators at the University of Exeter using an Agilent 620 FTIR microscope, an Agilent 670 FTIR spectrometer with a Global light source, and a liquid-nitrogen cooled 128x128 focal plane array detector [nallala'high-resolution'2016]. The spectral images produced by this process were then separated into stroma and non-stroma regions using K -means clustering methods and processed to remove paraffin spectral features as explained in [nallala'high-resolution'2016, mathieson'why'2020]. This work mostly focusses on the use of individual pixels within these spectral images for classification tasks. Each pixel contains FTIR absorption intensity data at 801 wavenumbers from 1000 cm^{-1} to 1800 cm^{-1} .

2.2 Cross-Validation Approach

FTIR data was produced for 90 samples, of which 31 were classified as normal, 17 as hyperplastic, 27 as adenomatous, and 15 as cancerous. Due to the desire to accurately predict performance in four-category prediction tasks, a 4-fold cross-validation approach was adopted [kohavi'study'2001]. This involved splitting the dataset into four parts using the stratified group K-fold method, which produces splits subject to the constraints of both preserving the proportion of samples (pixels) for each class within each split to the maximum extent possible whilst also enforcing the requirement for non-overlapping groups (images) between splits [t'r'stratifed'2023, gunopulos'stratification'2011, szymanski'network'2017]. When training ML models on spectral data, an approach was adopted where for each set of four folds, two were used for training, one for validation and one for testing. For methods which did not require validation or hyperparameter tuning, the validation set was discarded, enabling maximal comparability between methods.

2.3 Baseline Classification Methods

The most intuitive approach for classifying spectra according to their severity is to calculate the mean spectrum for each classification within the training set and then to measure which mean spectrum most closely resembles a test spectrum. Similar to work by Villazon et al. [villazon'cancer'2025], this work implements a mean square error (MSE) method, and a cosine similarity method. The mean square error is

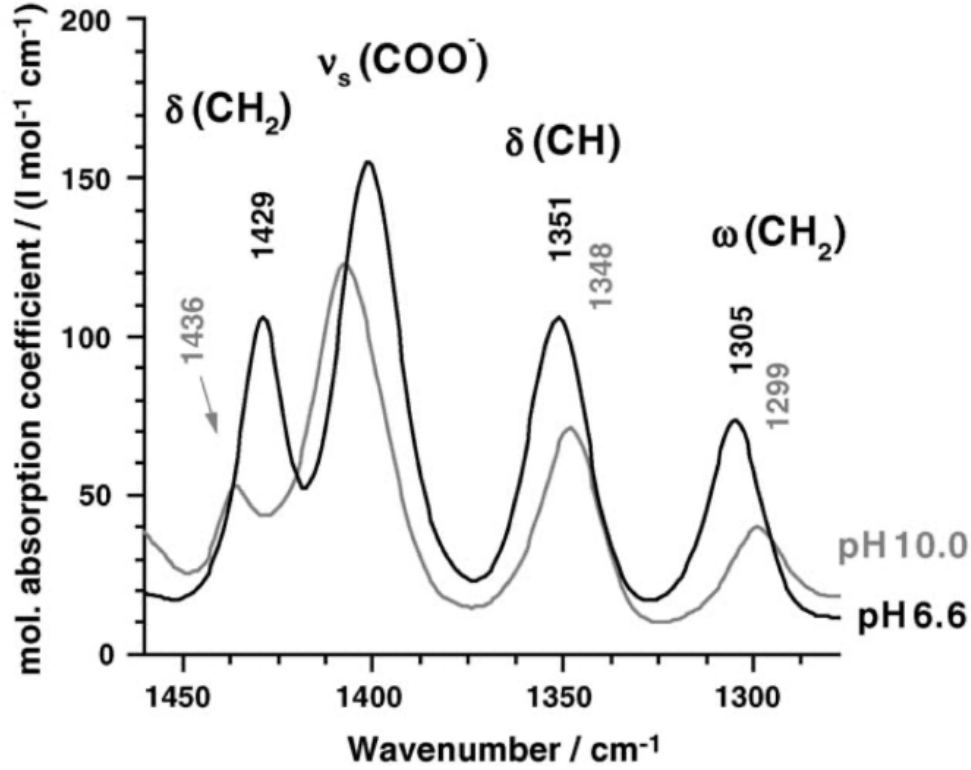


Figure 4: The FTIR spectra produced by cystine (an amino acid) at different pH levels [wolpert'infrared'2006].

defined as:

$$\text{MSE}(\vec{x}, \vec{y}) = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2 \quad (2)$$

where $\vec{x}$ and $\vec{y}$ are the spectra being compared, and n is the number of wavenumbers (dimensions) in each spectrum. The cosine similarity is also defined as:

$$\text{Cosine Similarity}(\vec{x}, \vec{y}) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| |\vec{y}|}, \quad (3)$$

which produces values between -1 and 1 , respectively denoting complete orthogonality and similarity between the two spectra. The classification of each test spectrum is then set as that which corresponds to the mean spectrum which minimises the MSE or maximises the cosine similarity.

2.4 Machine Learning (ML) For Classification

Throughout this work, I use three standardised machine learning processes for FTIR spectral classification, two of which utilise principal component analysis (PCA) (see ??) to reduce input data from 801 to 25 dimension. The methods implemented are PCA with linear discriminant analysis (LDA) [fisher'use'1936, ghojogh'linear'2019], PCA with extreme gradient boosting (XGBoost or XGB) [chen'xgboost'2016], and XGBoost applied directly to the 801 dimensional input data. Unless otherwise stated, the hyperparameters used for these methods take their default values as specified in the scikit-learn [pedregosa'scikit-learn'2011] and XGBoost [chen'xgboost'2016] libraries. A theoretical overview of these methods is also included in Appendix ??.

2.5 Neural Networks

Neural networks [schmidhuber'deep'2015, goodfellow'deep'2016, lussier'deep'2020] are machine learning algorithms built from layers of connected nodes (neurons) acted on by activation functions (§??). The weights and biases associated with each neuron are then learnt using backpropagation [rumelhart'learning'1986] and gradient descent [ruder'overview'2017] to minimise a loss function. This finds a mapping from inputs to outputs. With suitable model design, regularisation [goodfellow'deep'2016, kukacka'regularization'2017] and hyperparameter selection, these mappings can be tuned to generalise well to unseen data. Throughout this work, neural networks are trained, validated and tested on independent folds of data as described in §??, the Adam optimiser [kingma'adam'2017] is used for gradient descent, and hyperparameter tuning is conducted on the validation split using the Weights and Biases tool [biewald'experiment'2020]. A full overview on the different neural network layer types is included in Appendix ??, and a brief overview is included in Figure ??.

2.5.1 A Scale-Adjusting Convolutional Neural Network (SCNN)

In this work, a SCNN similar to that presented by Zaki et al. [zaki'explainable'2024] is implemented for feature extraction from 801 dimensional FTIR spectra. The model architecture is summarised in Figure ??. Notable features include the use of three scanning convolutional heads to extract information at different spectral scales, which is important for capturing the variable-width peaks in FTIR spectra. Furthermore, the use of tanh followed by ReLU activation functions reduces the effect of unphysically large convolutional outputs, which could otherwise produce exploding gradients [song'fine-grained'nodate, li'learning'2020, zaki'explainable'2024]. Finally, the last layers in the network learn a three parameter logistic output [yu'three-parameter'2021];

$$\hat{y} = c + \frac{(1 - c)}{1 + \exp(-a(\mathbf{x} - b))}, \quad (4)$$

where c is a guess parameter, b denotes difficulty and a denotes the discrimination. This allows the model to find a baseline offset for categorisation probability corresponding to the how it could sometimes be impossible to distinguish between classes using only FTIR data.

2.6 Explainability

2.6.1 Local Interpretable Model-Agnostic Explanations (LIME)

LIME [ribeiro'why'2016] is a model-agnostic explainability technique which works by producing N perturbations around an input vector, $\mathbf{x}$ and observing how perturbing different features alters classification probabilities. In this work, a variant of this is employed where each of the d input features (wavelength intensities) are one-hot encoded [samuels'one-hot'2024] into a vector, $\mathbf{x}'$ of length 10, with the encoded value corresponding to the training-set decile in which that feature value lies. Perturbed samples, $\mathbf{z}'$, are then generated by considering each feature, either retaining their one-hot encoding with probability 1/2, or uniformly allocating a new one-hot encoding, also with probability 1/2. A numeric vector $\mathbf{z}$ is then generated by randomly sampling within the deciles associated with each value in $\mathbf{z}'$. We then aim to train a local linear model $g \in \mathcal{G}$ to approximate the behaviour of the original model we wish to explain, f , around $\mathbf{x}$. Here, $\mathcal{G}$ is a class of potentially interpretable models. The selected LIME explanation is then given by:

$$\xi(\mathbf{x}) = \operatorname{argmin}_{g \in \mathcal{G}} [\mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g)], \quad (5)$$

where the locality-aware loss $\mathcal{L}(f, g, \pi_{\mathbf{x}})$ encodes the faithfulness of g to f in the locality $\pi_{\mathbf{x}}$ and $\Omega(g)$ is an explanation-complexity penalty term. This process is illustrated in Figure ??. In this work we are considering classification tasks, so the locality-aware loss is calculated using a square loss between the predicted probability for positive classification as given by f and g (the LIME process is repeated for each class). This takes the form:

$$\mathcal{L}(f, g, \pi_{\mathbf{x}}) = \sum_{\mathbf{z}, \mathbf{z}'} \pi_{\mathbf{x}}(\mathbf{z}) (f(\mathbf{z}) - g(\mathbf{z}'))^2, \quad (6)$$

where $\pi_{\mathbf{x}}(\mathbf{z}) = \exp(-D(\mathbf{x}, \mathbf{z})^2/\sigma^2)$ encodes locality through an L_2 [shan'targeted'2019, ribeiro'why'2016, why] distance function D and σ is generally set as the median of all D values. We then solve ?? using a

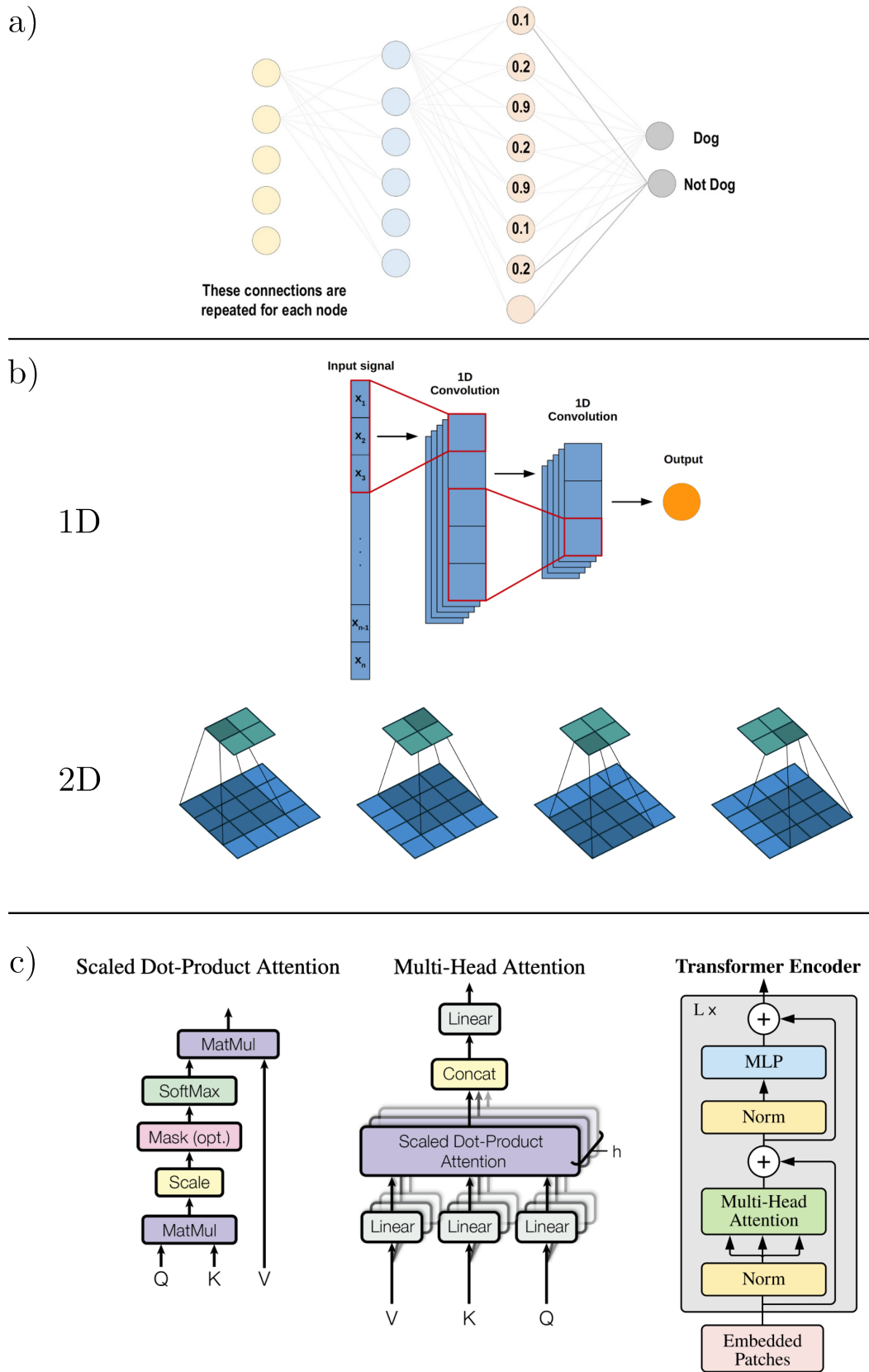


Figure 5: Different methods of connecting layers in a neural network. a) shows multiple fully-connected layers. From [alzubaidi'review'2021]. b) shows the action of 1D and 2D convolutional neural network (CNN) kernels. From [shenfield'novel'2020] and [dumoulin'guide'2018] respectively. c) shows an outline of the scaled dot-product attention and multi-head attention mechanisms, both from [vaswani'attention'2023], and the transformer encoder block, from [dosovitskiy'image'2021].

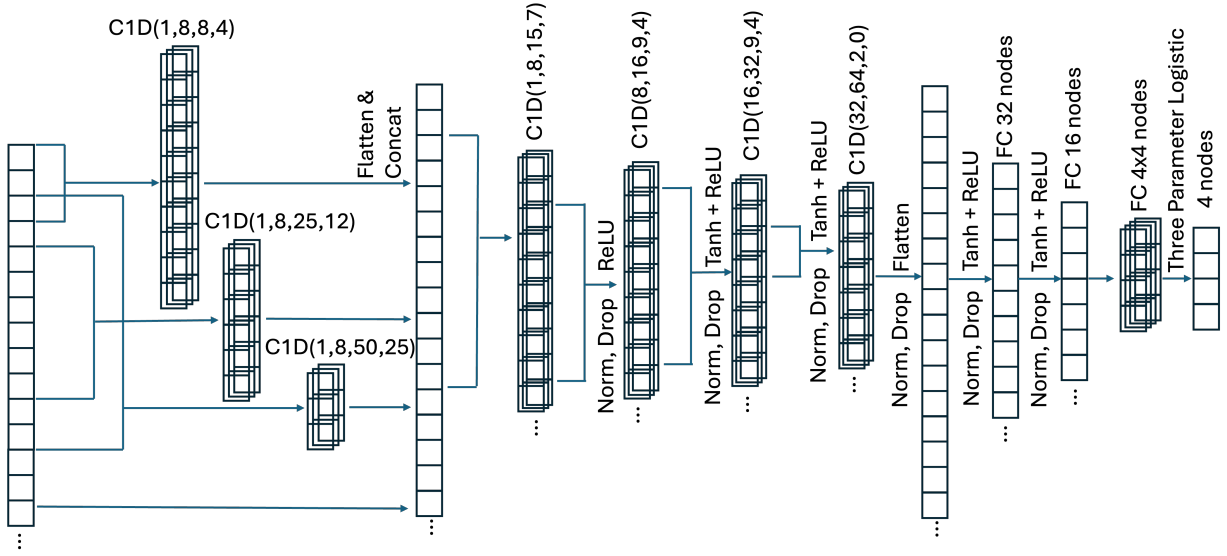


Figure 6: The SCNN architecture implemented in this work. 1D convolutional layers are represented by C1D(input channels, output channels, kernel size, stride) and fully connected layers are denoted by FC.

ridge regressor over N perturbed samples for each sample to be explained, giving [ribeiro'why'2016]:

$$(\mathbf{w}, \mathbf{b}) = \underset{\mathbf{w}, \mathbf{b}}{\operatorname{argmin}} \sum_{\mathbf{z}, \mathbf{z}'} \pi_x(\mathbf{z}) [f(\mathbf{z}) - (\mathbf{w}^T \mathbf{z}' + \mathbf{b})]^2 + \lambda \|\mathbf{w}\|_2^2, \quad (7)$$

where the weights $\mathbf{w} \in \mathbb{R}^{d \times 10}$ and biases $\mathbf{b} \in \mathbb{R}^d$ are those of the linear model g for a single class in a single sample. Calculating the L_2 norm of the 10 weights for each feature then produces relative importance scores for each feature within class k , $\mathbf{s}_k$. Finally, the total importance score for each feature is calculated by performing a probability-weighted sum over all classes [ribeiro'marcotcrlime'2025]:

$$\mathbf{s} = \sum_k p_k(\mathbf{x}) \mathbf{s}_k. \quad (8)$$

An example of the LIME weightings this process produces is shown in Figure ??.

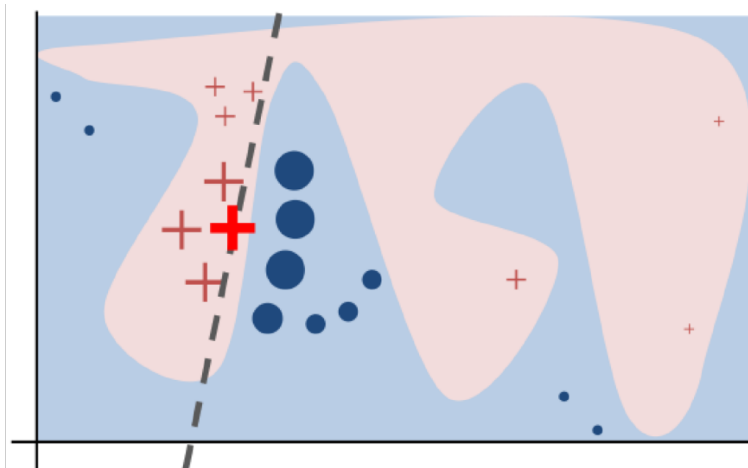


Figure 7: A demonstration of how LIME constructs a local linear classification model, applying greater weighting to nearby perturbations than distant ones as indicated by dot / cross sizes. From [ribeiro'why'2016].

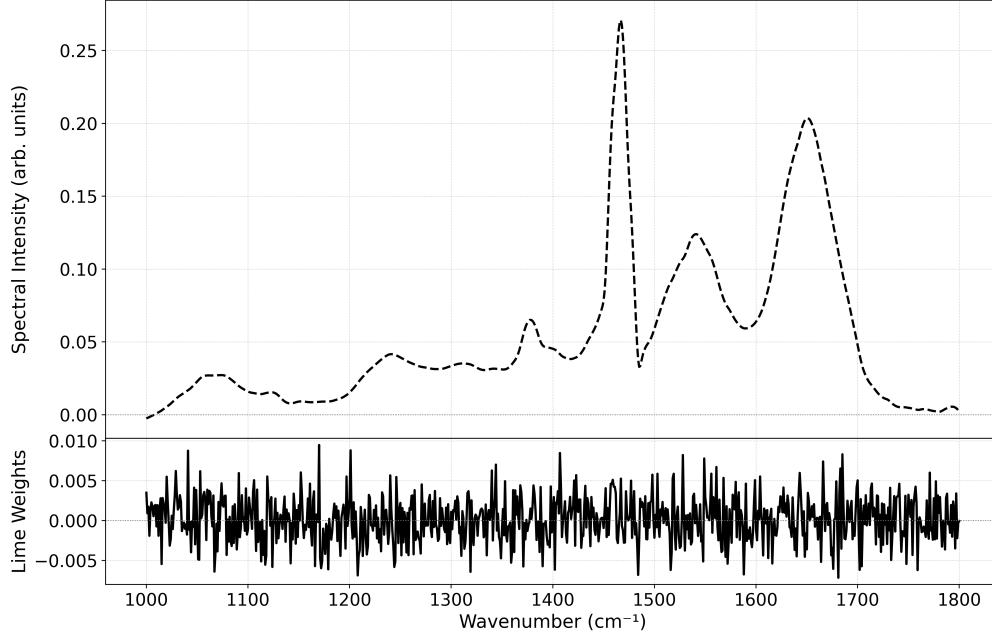


Figure 8: An example of the LIME weights produced for a single sample. The upper portion shows the original spectrum and the lower portion shows the relative importance of each feature for classification. Note how the weightings can be negative, indicating the feature acts against positive classification.

2.6.2 Context Representative Interpretable Model-Agnostic Explanations (CRIME)

The CRIME method [zaki’explainable’2024] aims to expand on LIME by finding contexts which have similar classification explanations. This is achieved by passing the LIME results through a variational autoencoder (VAE) [kingma’auto-encoding’2022, kingma’semi-supervised’2014] to learn a low-dimensional latent space representation of the explanations. This latent space is then clustered into K contexts using K -means clustering [jin’k-means’2011], each of which can then be investigated further.

2.6.3 Variational Autoencoders (VAE)

Attempting to find clusters of the explanations produced by LIME can be viewed as either an unsupervised learning problem - clustering the LIME spectra with no additional information, or as a semi-supervised learning problem, where known training labels are used to inform latent space construction. In this work, I implemented both approaches by implementing VAEs, which attempt to learn a generative model with parameters θ to enable mapping from the data $\mathbf{x}$ to a latent representation $\mathbf{z}$ and back again - illustrated in Figure ???. A theoretical overview of VAEs is included in Appendix ??, and shows that an unsupervised VAE is optimised by minimising the negative evidence lower bound (ELBO):

$$\mathcal{L}(\mathbf{x}) = -\frac{1}{2}\text{MSE}(\mathbf{x}, \mathbf{y}_\mu) + \beta \frac{1}{2} [1 + \log(\sigma^2) - \mu^2 - \sigma^2], \quad (9)$$

where $\mathbf{y}_\mu$ is the output of the decoder, β is a hyperparameter controlling the relative weighting of the reconstruction loss and KL divergence terms [higgins’beta-vae’2017], and μ and σ^2 are the latent space mean and variance projections respectively.

When using a semi-supervised VAE approach, we follow the M2 method described by Kingma et al. [kingma’semi-supervised’2014] so modify (??) to include a classification cross-entropy loss [kingma’semi-supervised’2014, mao’cross-entropy’2023] assessing the difference between predicted and true class probabilities. These predicted class probabilities are given by softmax over the output from a two-layer (64 $\rightarrow$ 128 $\rightarrow$ 4 node) fully-connected layer at the end of the VAE encoder, prior to μ and σ projection. The implementations of these ELBOs and the reparametrisation trick can be found in §?? and §?? respectively. The VAE architectures used in this work are summarised in Table ?. Furthermore, because each VAE is already implicitly using a model trained on the training set for each split, the VAE itself is trained on the LIME explanations of the validation set.

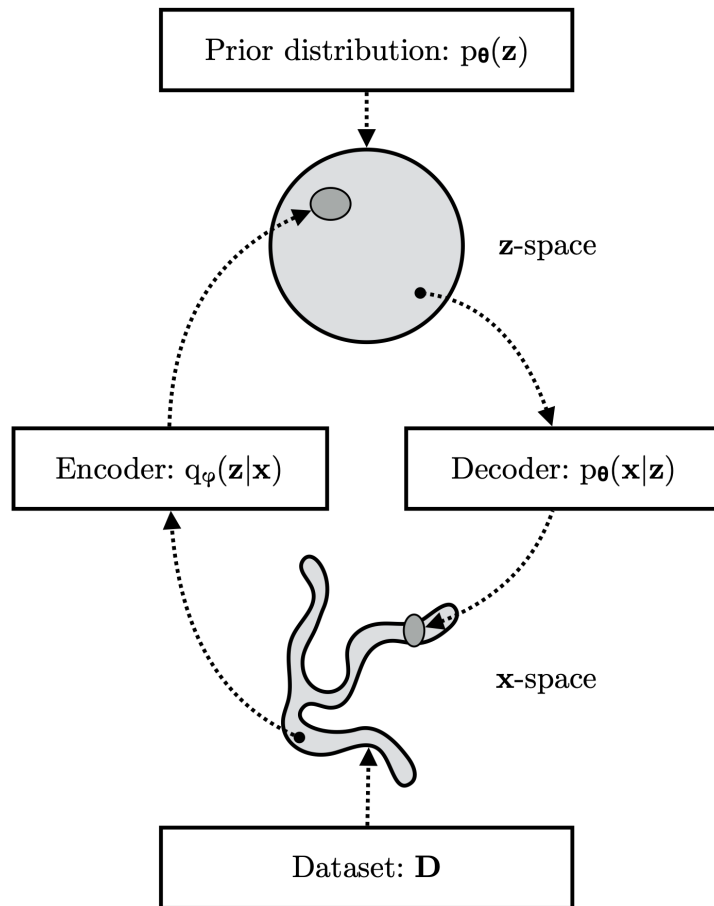


Figure 9: An illustration of the mapping from input values to a latent space and back to prediction values within a VAE. From [kingma'introduction'nodate]

Table 2: Architectures of the three semi-supervised VAE methods studied. BN = BatchNorm1d, C1D(ic, oc, k, s) = Conv1d(input channels = ic , output channels = oc , kernel = k , stride = s), FC = fully-connected (bias on), TE(D)(d, nh, f, dr, nl)= TransformerEncoder(Decoder)(model dim = d , num heads = nh , feed-forward dimension = f , dropout = dr , num layers = nl).

Model	Feature extractor	Classifier head	Latent & posterior path	Decoder
Convolutional - Perceptron (CP)	BN($input\ dim$) $\rightarrow$ [C1D(1,1,3,2); BN; ReLU] = seq ₁ , [C1D(1,1,300,50); BN; ReLU] = seq ₂ ; Concat(x_{raw} , seq ₁ , seq ₂) $\rightarrow$ FC($flat\ len \rightarrow 64$); ReLU	FC(64 $\rightarrow$ 128); ReLU $\rightarrow$ FC(128 $\rightarrow$ 4)	Concat($[h, y]$) $\rightarrow$; FC(68 $\rightarrow$ 4) = μ , FC(68 $\rightarrow$ 4) = $\log \sigma^2 \rightarrow$ $z = \mu + \epsilon \sigma, \epsilon \sim \mathcal{N}(0, 1)$	BN(4 + 4) $\rightarrow$ FC(4 + 4 $\rightarrow input\ dim$) = dec; tanh(dec) $\times m + c$ Learned scalars m and c
Convolutional - Transformer (CT)	<i>Same as above</i>	<i>Same as above</i>	<i>Same as above</i>	[FC(4 + 4 $\rightarrow$ 64) = proj], Rand($input\ dim, 64$) = query, causality] $\rightarrow$ TD(64, 4, 256, 0, 3) $\rightarrow$ FC(64 $\rightarrow$ 1) = dec; tanh(dec) $\times m$ Learned scalar m <i>Same as above</i>
Transformer - Transformer (DT)	BN($input\ dim$) $\rightarrow$ [C1D(1,1,3,2); BN; ReLU]= seq ₁ , [C1D(1,1,300,50); BN; ReLU]= seq ₂ ; Concat([seq ₁ , seq ₂]) $\rightarrow$ Token project FC(1, 4) + Sinusoidal PE $\rightarrow$ TE(4, 4, 64, 0.1, 3); AdaptiveAvgPool $\rightarrow$ vector $\rightarrow$ FC(4 $\rightarrow$ 64); ReLU	<i>Same as above</i>	<i>Same as above</i>	<i>Same as above</i>

2.6.4 CRIME with Linear Classification

Once the CRIME method has been used to cluster the LIME explanations into contexts, the mean FTIR spectrum associated with the LIME explanations in each context can be found. This work then uses these mean spectra, along with the mean LIME weights for each context to produce an explainable linear classification model. This is achieved by first classifying each context according to the most common classification (normal, hyperplastic, adenoma, or cancer) of the spectra which are placed into the context when training the VAE on the validation set. We then compute a LIME-importance-weighted cosine similarity between each mean cluster spectrum and a test spectrum. The method used to calculate this similarity was initially developed by Zaki et al. to calculate the similarity of their clustered spectra to known indicator spectra [zaki'explainable'2024]. For each clustered spectrum, μ , with weights w , we scale both to a range $[-1, 1]$ according to the minimum and maximum values within each vector, giving scaled vectors $\hat{\mu}$ and $\hat{w}$. We then do the same for a test spectrum $t \rightarrow \hat{t}$. The cosine similarity is then calculated as:

$$\text{sim} = \frac{\hat{t}^T (\hat{\mu} \cdot \hat{w})}{|\hat{t}|^2 |\hat{\mu} \cdot \hat{w}|^2}. \quad (10)$$

Each test set spectrum is then classified according to the context with which achieves the highest similarity score. The respective context classification is then used to map the test spectra to a physical classification (normal, hyperplastic, adenoma or cancer). Due to the low number of independent samples, it is not possible to further divide the dataset to find a reliable validation set for this method, so sensible VAE parameters are selected based on their performance classifying the validation set before performing a final run to evaluate performance on the test set.

2.7 Evaluation Metrics

The dataset used in this work is highly imbalanced, so a very high accuracy can be achieved by simply predicting the most common class. To avoid this, I calculated performance metrics relative to each "true" classification, k , using the following confusion matrix (CM) (see Figure ??) conventions:

$$\text{TP}_k = \text{CM}_{kk}, \quad \text{FN}_k = \sum_j \text{CM}_{kj} - \text{TP}_k, \quad (11)$$

$$\text{FP}_k = \sum_i \text{CM}_{ik} - \text{TP}_k, \quad \text{TN}_k = \sum_{i,j} \text{CM}_{ij} - (\text{TP}_k + \text{FN}_k + \text{FP}_k). \quad (12)$$

From these counts we then define:

$$\text{Sensitivity}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \quad \text{Specificity}_k = \frac{\text{TN}_k}{\text{TN}_k + \text{FP}_k}, \quad (13)$$

$$\text{Accuracy}_k = \frac{\text{TP}_k + \text{TN}_k}{\text{TP}_k + \text{TN}_k + \text{FP}_k + \text{FN}_k}, \quad \text{FPR}_k = \frac{\text{FP}_k}{\text{FP}_k + \text{TN}_k}. \quad (14)$$

Furthermore, to fairly measure performance despite dataset imbalance, we define the overall weighted accuracy (OWA) as:

$$\text{Overall Weighted Accuracy} = \frac{\sum_{k=0}^K w_k \cdot \text{Accuracy}_k}{\sum_{k=0}^K w_k}, \quad (15)$$

$$\text{where } w_k = \frac{\text{True number of samples in class } k}{\text{Total number of samples}}. \quad (16)$$

2.7.1 Binary Classification Methodology

Due to the high computational cost in training the SCNN networks, it was decided that binary classification performance results throughout this work would be calculated using four-class classification predictions. For this, we categorise cancer, hyperplastic and adenomatous predictions as non-cancerous and cancerous predictions are unchanged.

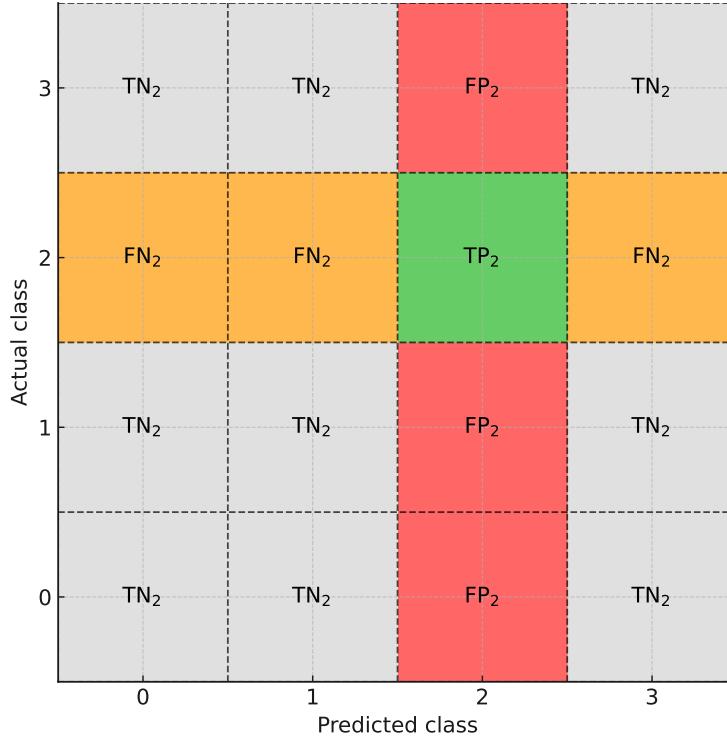


Figure 10: A confusion matrix showing the contributions to true positive (TP), false negative (FN), false positive (FP) and true negative (TN) counts for a sample "truly" belonging to class 2.

3 Results and Discussion

3.1 Dataset Analysis

The data for this work was collected as described in Section ??, resulting in a dataset containing 90 colon biopsy samples, their associated H&E stained images, their FTIR spectral images with absorption intensities at 801 wavenumbers from 1000-1800 cm^{-1} , and their histopathological classification as either normal, hyperplastic, adenomatous or cancerous tissue [nallala'high-resolution'2016]. Notably, this histopathological classification was made according to the most severe tissue condition present within the sample, so a sample containing only 2-3% cancerous tissue would still be classified as cancerous. We therefore expect many individual pixels within the spectral images to be incorrectly labelled. This could contribute to the very similar average spectra in all classes as shown in Figure ??.

False-colour images for samples were generated by taking the intensities for each pixel at high-variance wavenumbers 1080 cm^{-1} , 1230 cm^{-1} and 1600 cm^{-1} , and then scaling them to values between 0 and 1, centred around 0.5 according to the standard deviation of intensity values at that wavenumber throughout the sample. For each classification, one of these false-colour images along with its associated H&E stain is presented in Figure ?. This figure both demonstrates the boundary of stroma regions and the locations of H&E image folds preventing perfect alignment. Only the non-stroma regions are used for subsequent prediction analysis.

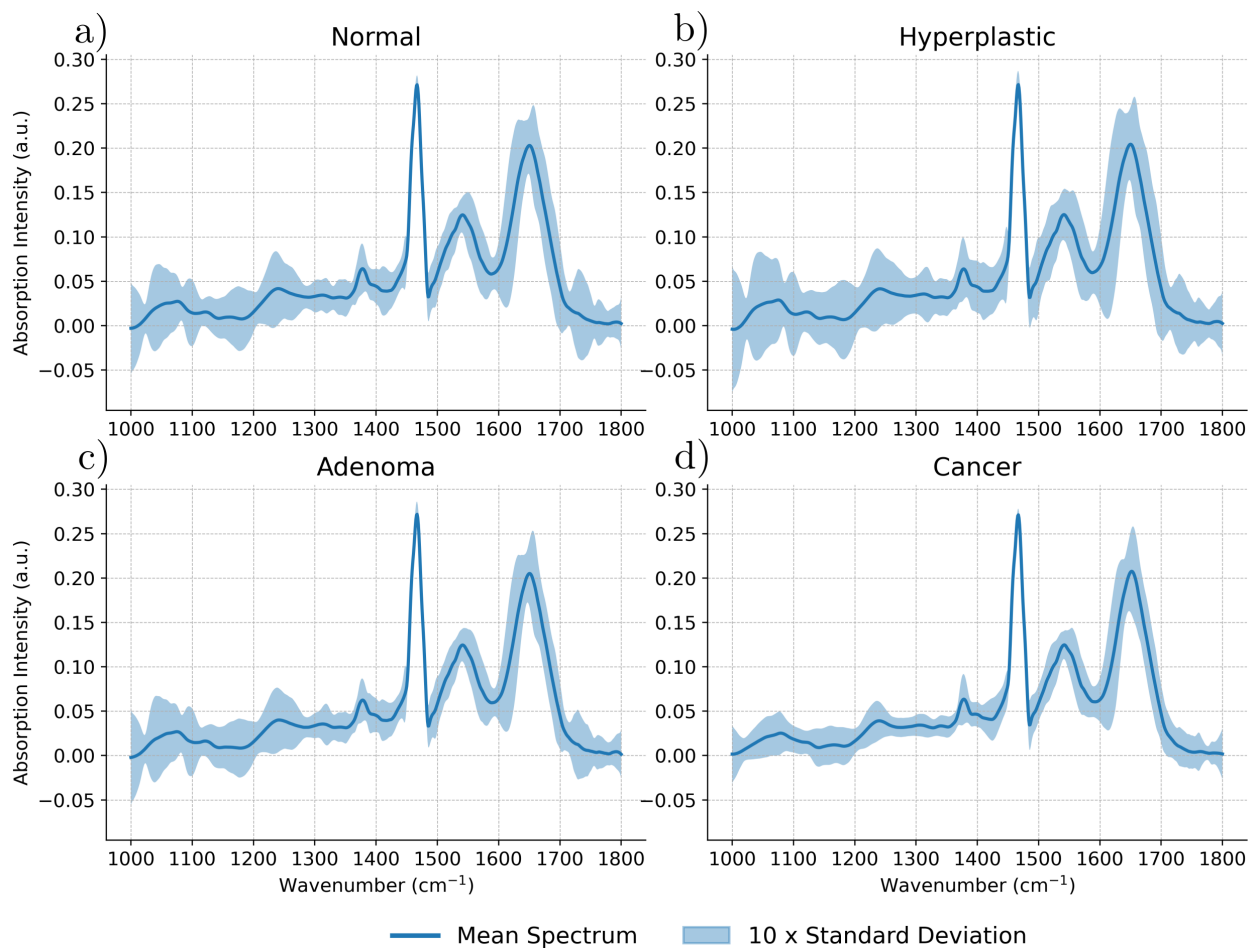


Figure 11: Plots showing the mean and standard deviation of the FTIR spectral intensities across each class. The cancerous sample spectra are seen to have slightly smaller standard deviations than other classifications. The only visually discernable difference between the mean spectra is around 1120 cm^{-1} , where the cancerous spectra does not show a small peak which can be observed in the spectra associated with the other classes.

3.2 Classification Methods

3.2.1 Traditional Classification Methods

Each of the classification methods described from §?? to §?? were applied to the dataset to produce both four-class and binary classification performance results. In all cases, the default hyperparameter values were used. The performance results from these experiments are presented in Tables ?? and ?? respectively. These tables also show the performance of SCNN method applied in this work for comparison.

3.3 A Scanning Convolutional Neural Network Approach

The SCNN architecture described in §?? was trained on each dataset fold (§??) using a weighted cross-entropy loss to predict the 4-class classification of each spectral pixel. Significant regularisation and hyperparameter turning were required to achieve reliable model convergence. For this tuning, random search was employed to identify hyperparameter ranges which consistently produced converging models. These ranges are presented in Table ?. Random search was then again employed within each of these hyperparameter ranges to produce a set of converged models. Figure ?? shows the OWAs these models achieve on each dataset fold. The difficulty in achieving model convergence is likely due to the incorrect labelling of many pixels in the dataset. It is therefore thought that future works could investigate the use of an ordinal loss function [castagnos’simple’2022, polat’class’2025] to additionally penalise predictions more severe than the true

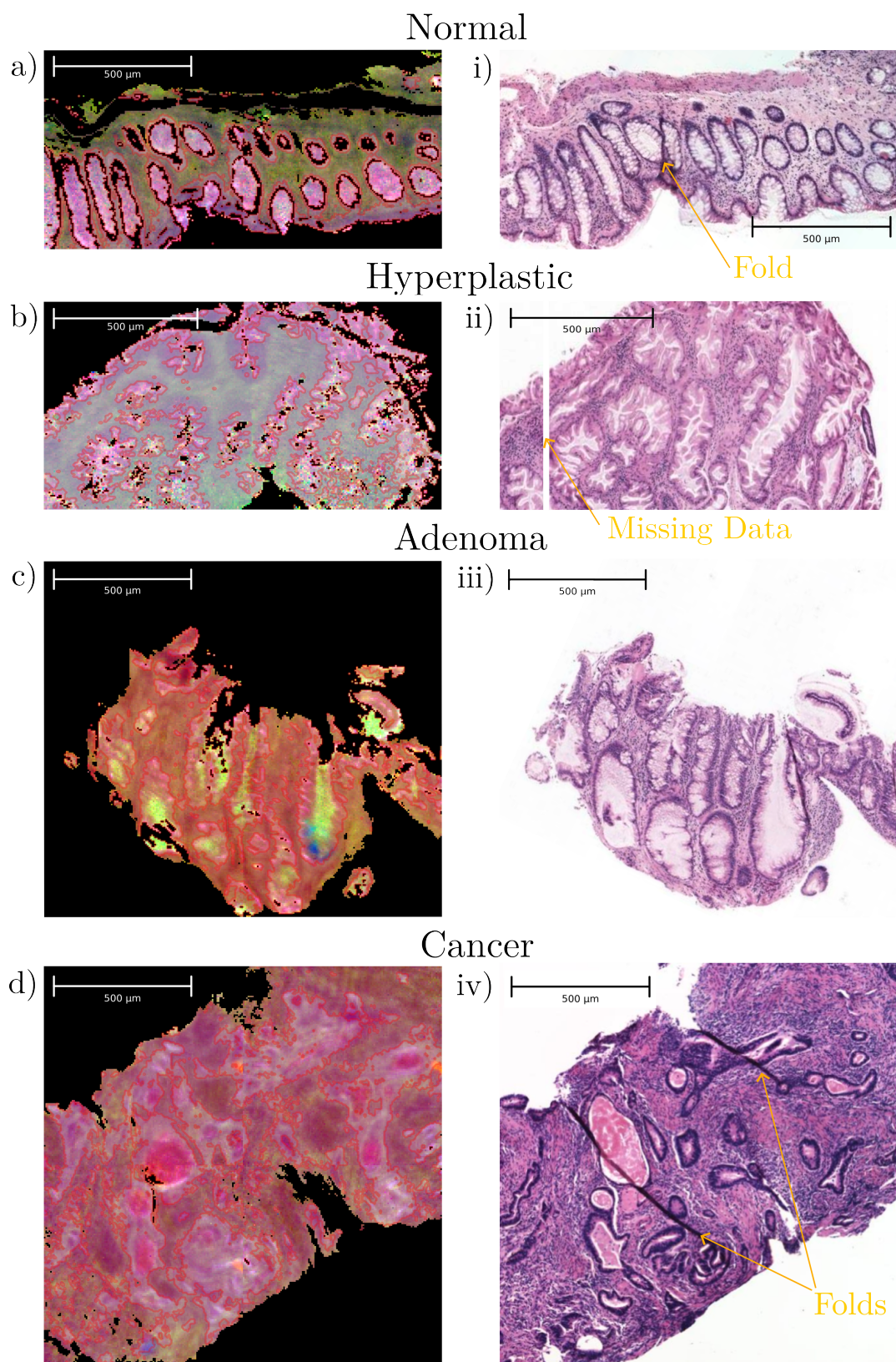


Figure 12: Sections a-d show false-colour FTIR images of colon biopsy samples - one per labelling classification. Each of these has non-stroma regions surrounded by red outlines and highlighted with a white accent. Sections i-iv show the corresponding aligned H&E images. Note that the H&E images in sections i and iv show folding defects, and information for a small stripe of the image in section ii is missing, so is blanked.

label.

Parameter	Distribution	Numeric Bounds
Dropout Rate	Log-uniform	0.01 $\rightarrow$ 0.30
Adenoma Entropy Weight	Uniform	0.05 $\rightarrow$ 0.40
Leak Rate	Log-uniform	$1 \times 10^{-6} \rightarrow 0.80$
Learning Rate	Log-uniform	$1 \times 10^{-6} \rightarrow 0.05$
Scheduler Step Size	Integer uniform	5 $\rightarrow$ 20
Weight Decay	Log-uniform	$1 \times 10^{-5} \rightarrow 0.50$

Table 3: Tuned hyperparameter ranges used for SCNN model training.

Because model convergence was still unstable within this tuned hyperparameter range, it was decided to apply an ensemble approach to the SCNN predictions, where all models trained on the same fold with validation set OWA scores above the median value in the worst performing fold (taken to be 55%) were used to make predictions on the test set. These selected models were combined using a voting approach, where the predicted class with the highest number of votes was selected as the final prediction. The performance of this method for four-class and binary classification in the test set is presented in Tables ?? and ?? respectively, along with the performance of the traditional machine learning methods described previously.

It was hypothesised that label smoothing [xia'towards'2025] could improve model performance by encoding a small amount of uncertainty in the training set labels. Due to the high computational cost of training these full ensemble SCNN models, this was tested only on dataset fold "a" (as labelled in Figure ??). The test set performance results of these trials are presented in Table ??, and show that increasing label smoothing from 0% to 20% slightly reduces OWA scores. It should be noted, however, that these results cannot be used to inform the SCNN method trialled here because they would introduce test-set information into the model. Label smoothing could therefore also be investigated further by future works.

Smoothing	Metric	Normal	Hyperplastic	Adenoma	Cancer
0%	Sensitivity	0.662	0.258	0.452	0.783
	Specificity	0.708	0.762	0.936	0.972
	Accuracy	0.699	0.680	0.699	0.945
	False Positive Rate	0.292	0.238	0.064	0.028
	Overall Weighted Accuracy	0.5387			
10%	Sensitivity	0.643	0.257	0.458	0.782
	Specificity	0.716	0.755	0.933	0.971
	Accuracy	0.702	0.674	0.700	0.944
	False Positive Rate	0.284	0.245	0.067	0.029
	Overall Weighted Accuracy	0.5349			
20%	Sensitivity	0.650	0.266	0.436	0.786
	Specificity	0.713	0.751	0.937	0.967
	Accuracy	0.700	0.672	0.692	0.941
	False Positive Rate	0.287	0.249	0.063	0.033
	Overall Weighted Accuracy	0.5345			

Table 4: Fold "a" test-set SCNN classification performance with a 55 % voting threshold and different label-smoothing levels.

3.4 SCNN Performance Investigation

The performance of the SCNN method beats all other methods in both binary and 4-class classification, as shown in Figure ??, achieving weighted overall accuracies of 0.832 ± 0.059 and 0.587 ± 0.037 respectively. I therefore investigated the ensemble SCNN predictions further to discover where this model performs well and where it does not. The summed confusion matrix for 4-classification across the four folds is shown in Table ?. From this, we see that the model is best at classifying normal and cancerous tissue, but predicts

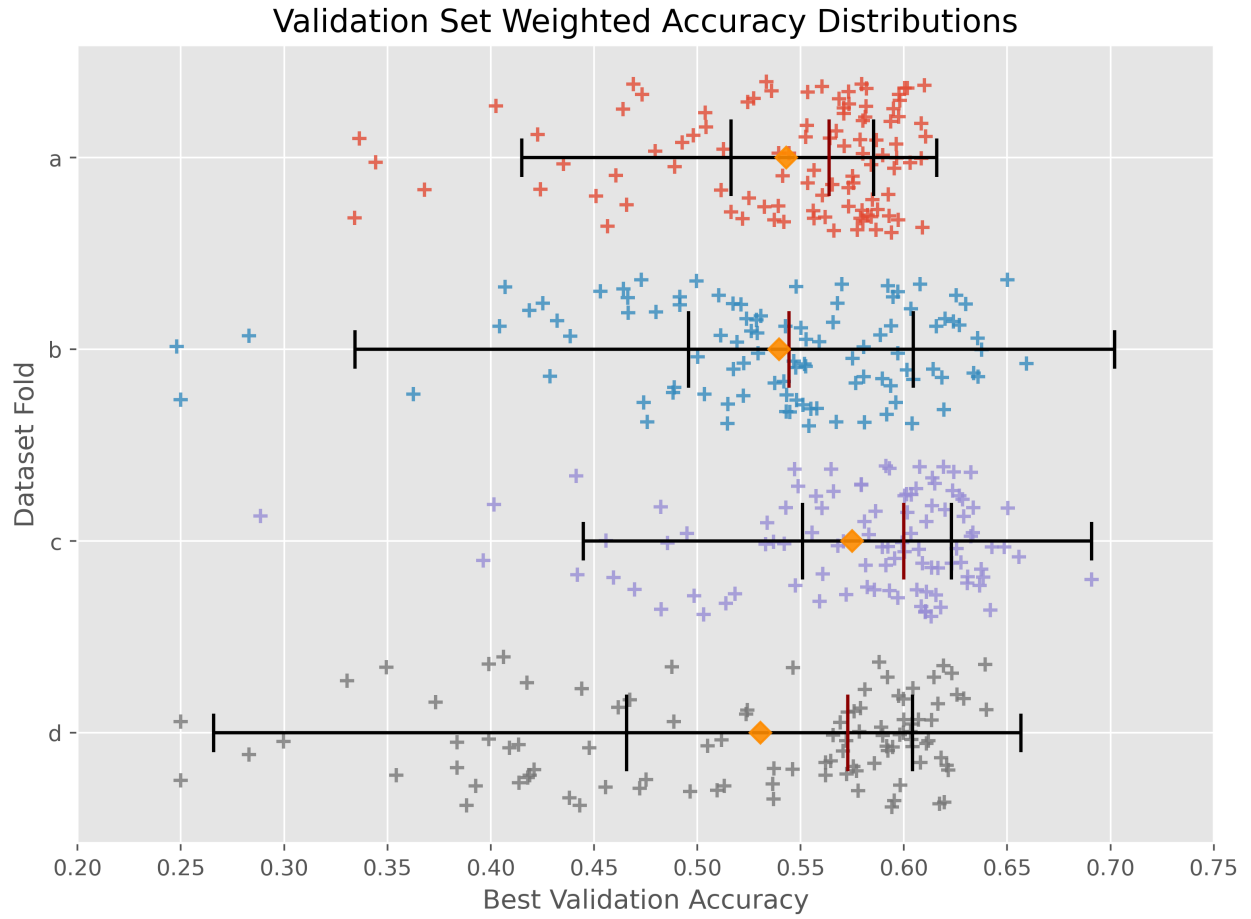


Figure 13: Validation set OWA performance of the SCNN models trained on each dataset fold. Ticks are shown at ends of the first and third quartiles, Q_1 and Q_3 , the median (red) and at either $1.5 \times$ the inter-quartile range from Q_1 and Q_3 or the maximum / minimum recorded value, whichever is closest to the median. The mean values are denoted by orange diamonds.

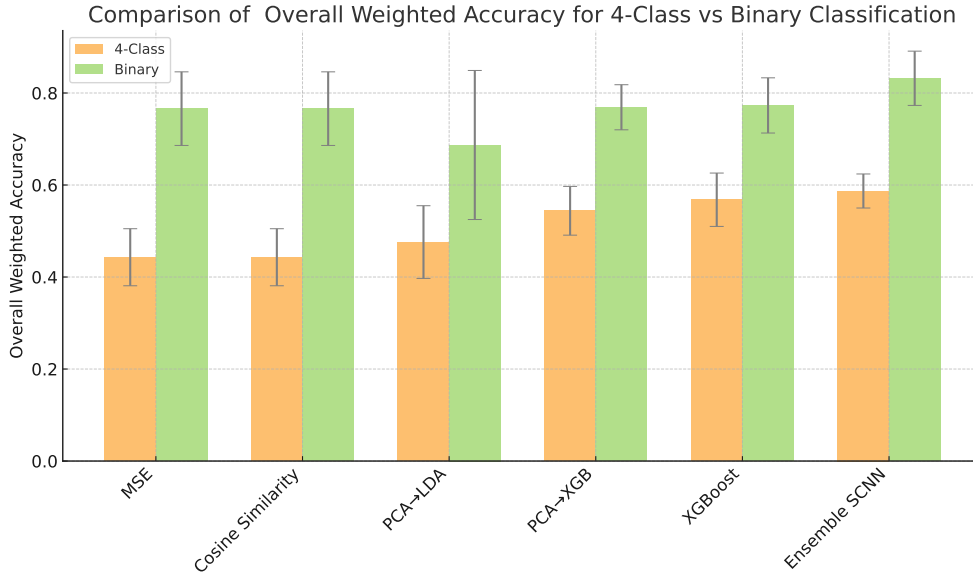


Figure 14: The OWA achieved by each ML method, and the SCNN method both for binary and 4-class classification. The SCNN method is seen to perform best in both cases.

hyperplastic tissue to be normal more often than not. It also frequently predicts adenomatous tissue to be normal or cancerous. These results, however, are somewhat expected and in fact desired given the sample classification method used (§??). This implies there is a limit to the usefulness of the comparative metrics. I therefore also conducted qualitative analysis with the assistance Dr. Nalalla of the University of Exeter. Some of the SCNN predictions analysed are presented in Figure ?? . Here, we see that the model refuses to fit to some samples within the training set (Figure ??c), strongly indicating that the model is learning to classify pixels better than the classifications it is provided with to train.

To further demonstrate how this performance is superior to that suggested by the classification statistics, we can compare the H&E images in Figure ??i-iv with the SCNN predictions in Figure ??i-iv. For the sample presented in the i) sections, we see that the SCNN model has predicted some normal tissue to be hyperplastic, and some to be cancerous. This is thought to be due to the presence of inflamed tissue in the sample, which is best described as an additional classification category. The sample presented in the iii) sections clearly shows how the model has learnt to correctly differentiate between adenomatous and normal tissue. Furthermore, the predictions of hyperplasticity in the iv) sections are thought to be due to the presence of necrotic tissue in the sample, which again is best described as an additional classification category. Future works could therefore investigate the application of this model to more finely labelled datasets, or could try incorporating more samples in the training set to provide labelled information on inflamed and necrotic regions.

To complement this analysis, I also investigated the confidence the model has in its predictions by evaluating the number of votes each class receives in the ensemble prediction for the same pixel. The results of this analysis for the same samples as were previously discussed are presented in ?? . In section ??iv, we see that the model is particularly uncertain around the transition from cancerous to hyperplastic predictions. This is expected, as such regions are unlikely to have sharp boundaries. Furthermore, in section ??iii, we see that it is less confident in predicting tissue to be normal than adenomatous, which is again likely due to diffuse boundaries which should really be considered as containing cells intermediate to either classification.

3.4.1 LIME Explanations

LIME analysis was also conducted on a subset of the SCNN validation set predictions - 5000 pixels per classification type (grouped by their training label). This allowed for the generation of importance weightings for each wavenumber and prediction classification. These can be seen in Figure ?? . These results indicate that the region from $1000-1200\text{ cm}^{-1}$ is particularly relevant for all classifications, and that the region from

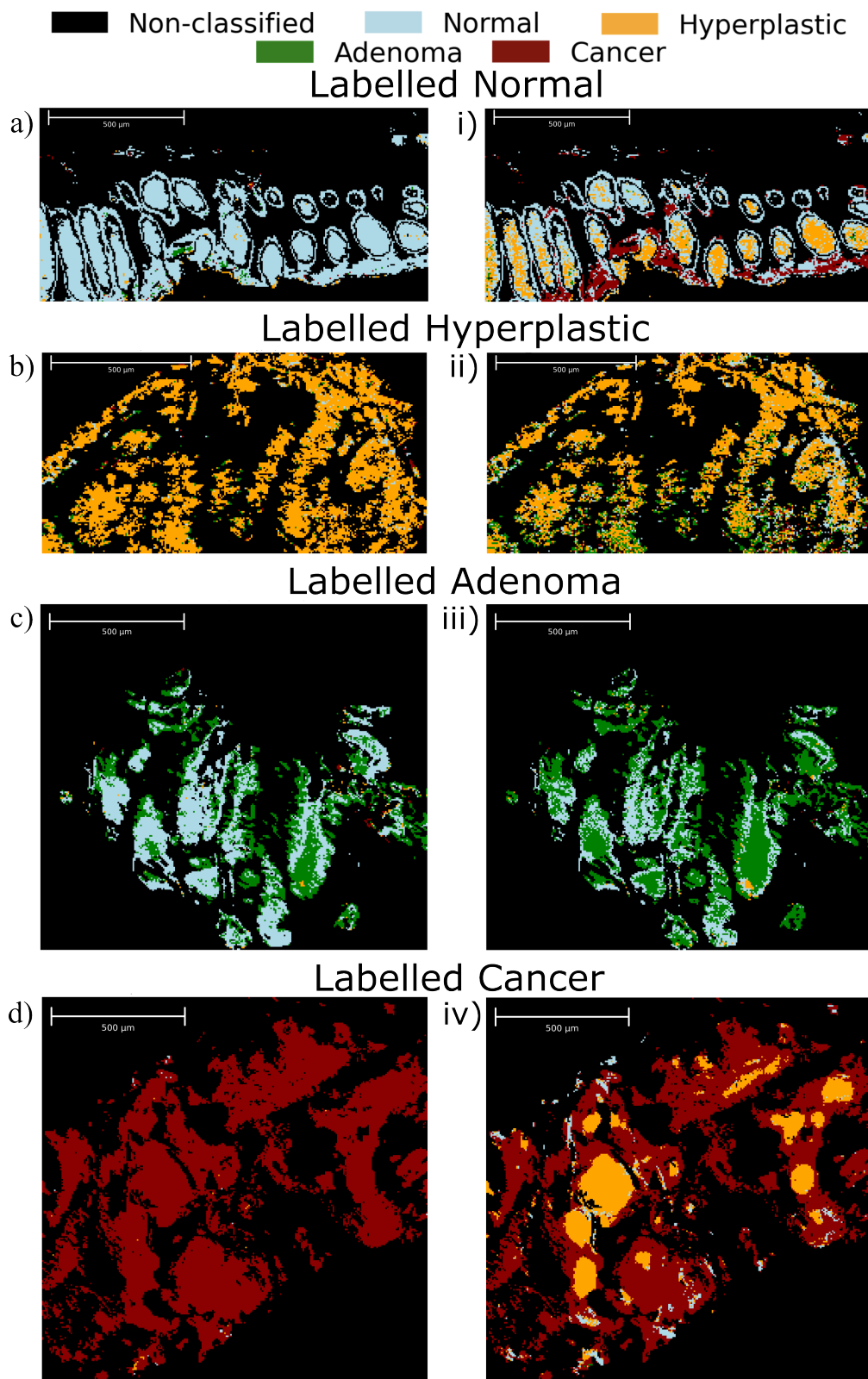


Figure 15: SCNN classification predictions for the same samples when each is used in the training set (a-d) and when each is used in the test set (i-iv).

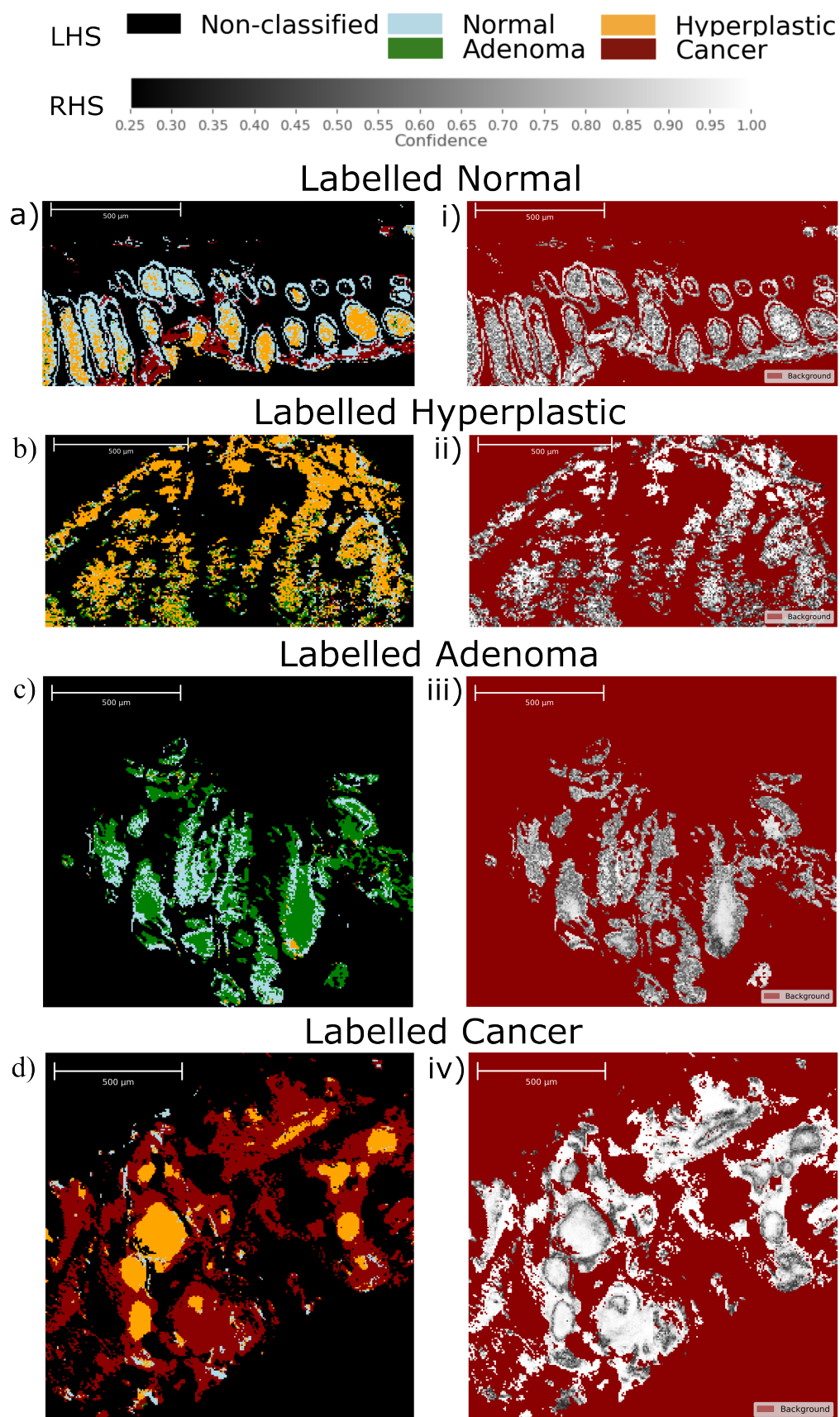


Figure 16: The confidence of the SCNN model in its classification predictions for each sample considered in Figures ?? and ??.

Labelled \ Predicted	Normal	Hyperplastic	Adenoma	Cancer
Normal	164993	28191	31533	7890
Hyperplastic	77386	62551	29068	23767
Adenoma	124631	69665	330515	68378
Cancer	9588	14258	17966	132174

Table 5: Summed confusion matrix (rows = true class, columns = predicted class).

1460-1560 cm^{-1} is much less important. This could be used to help guide the development of next generation imaging methods such as those being developed as part of the TROPHY project [noauthor'ultrafast'2022].

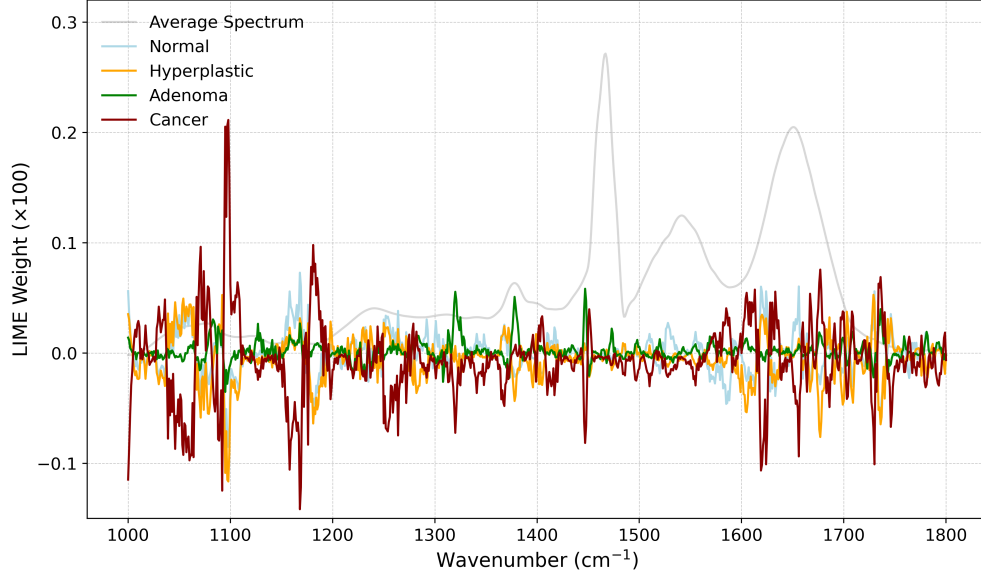


Figure 17: LIME analysis results showing the average LIME importance associated with each wavenumber feature.

3.5 CRIME Analysis

The LIME explanations found in the previous section enable the generation of explanations for any individual prediction. However, they do not provide a good understanding of how the model work as a whole. to address this, the CRIME method described in ?? was applied using the 20,000 pixel explanations generated for §??. For this, each VAE network was trained on the validation set pixels. Throughout this training, the α and β hyperparameters were tuned such as to generate distinct clusterings in latent space. The used were $\alpha = 1000$, $\beta = 0.1$ for the CP and DT VAEs, and $\alpha = 1000$, $\beta = 1 \times 10^{-3}$ for the CT VAE. The contexts were then formed by K-means clustering with $K = 100$ in all cases. An example the latent space representation and the associated contexts for $K = 20$ produced by the unsupervised CP VAE are shown in Figure ??. This demonstrates how the LIME explanations can be used to effectively separate tissue classifications in latent space.

To quantify this separability, I calculated the SCNN ensemble prediction associated with each explanation. For each context, I then counted the number of these predictions associated with each tissue classification. The same was done using the training label associated with each explanation. In the $K = 100$ case applied to the latent space representation found in Figure ??b, this generated the histograms shown in Figure ??. A careful analysis of these histograms shows that the cancer dominated contexts are associated with very few non-cancer predictions, and the same is also true for all of the dominant peaks in the other classifications. I therefore hypothesise that this method has found some independent mappings from explanations into tissue classifications. To investigate this further, I classified each context according to the majority SCNN ensemble classification associated with the explanations within that context. By then finding the average spectra

and LIME weightings associated with each context, I was then able to form multiple explained spectra for each classification. Figure ?? shows a few examples of these explained context spectra for the same setup as in Figures ?? and ?. Here, we see that although the average spectra for all contexts are similar, the LIME weightings are visibly different, indicating that the model is using different spectral features to make predictions in each context. Further work could therefore be done to investigate the biological and physical significance of these context importance weightings - a task which is feasible, unlike trying to interpret the LIME weightings for each explanation individually.

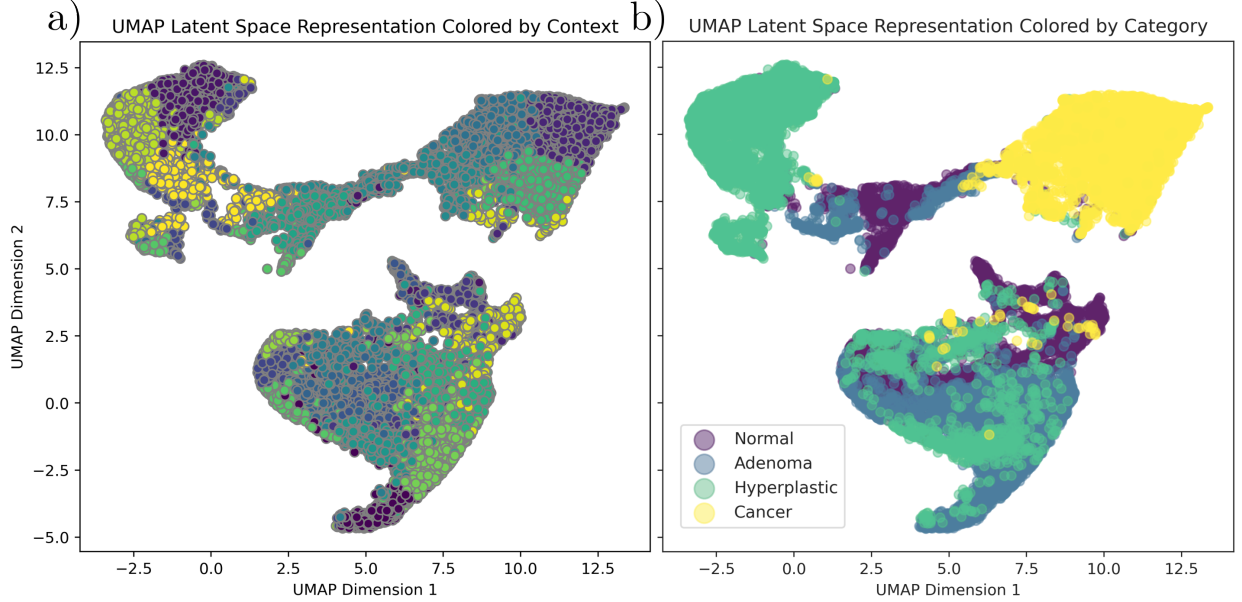


Figure 18: Section a) shows the latent space representation of the LIME explanations generated using the CP VAE. Each colour denotes a different context designated by K-means clustering. Section b) shows the training label associated with each explanation projected into latent space.

3.5.1 CRIME Predictions

The CRIME linear prediction method (LCRIME) described in §?? was then employed using the majority classified context spectra and weightings. It was noted, however, that although the CRIME method with an unsupervised CP VAE is able to separate cancer and hyperplastic tissues well, many contexts have multiple similarly-frequent classifications present. To improve this, the M2 semi-supervised versions of the VAEs described in Table ?? were also implemented to encourage the latent space to be more separable. The resulting four-class and binary classification prediction performance metrics achieved by each VAE model are described in full in Tables ?? and ?? respectively. Of these, the best metrics for comparison are shown graphically in Figures ?? and ?? for four-class and binary classification tasks respectively. These results show that the LCRIME method generally performs worse than all other methods tested at training-label prediction. The best LCRIME models (CP-0.2 and CT-0.2) are, however, able to reconstruct the model four-class classification predictions better than the MSE and cosine similarity classification are able to reproduce the training labels. LCRIME also achieves a similar performance to the PCA→LDA method by this metric. LCRIME is, however, always out-performed by the PCA→XGB, XGB and ensemble SCNN methods. Despite this, LCRIME still offers advantages because it is intrinsically interpretable and the method by which it makes prediction is easy to explain to patients and clinicians.

Future works could investigate the use of the LCRIME method on less-noisy datasets, or could try to perform more thorough hyperparameter searching by folding the dataset differently. It would also be informative to compared the performance of this LCRIME method against a directly learnt linear mapping model.

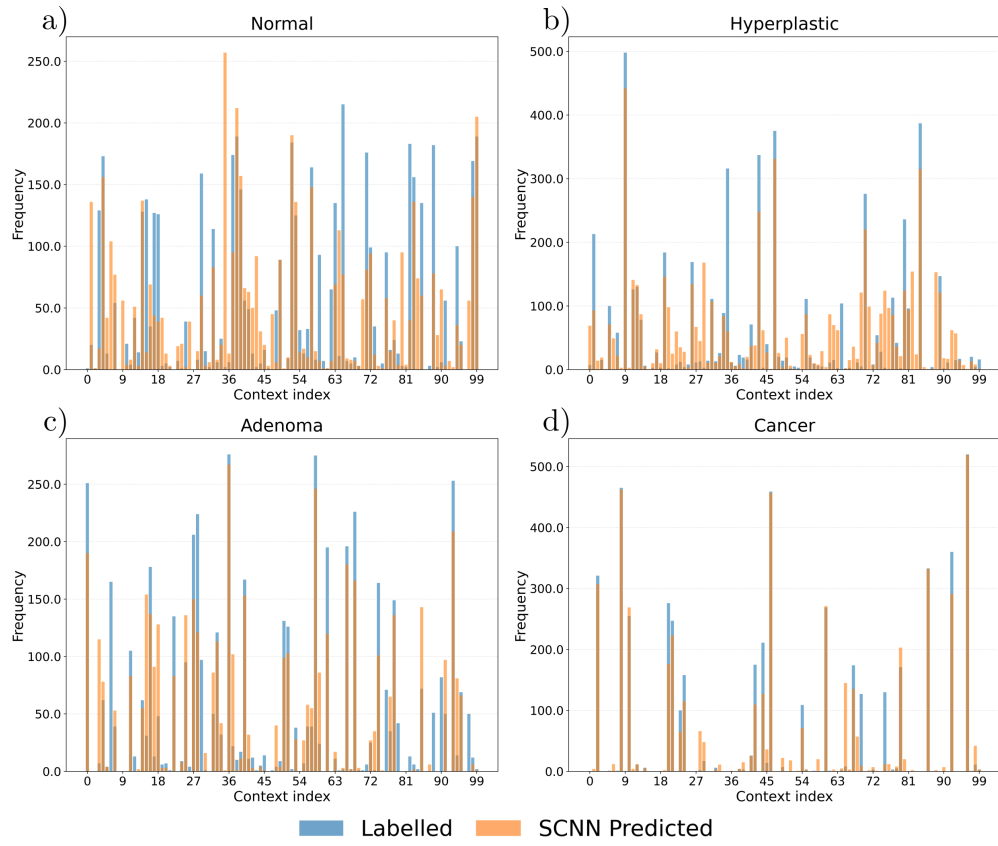


Figure 19: The number of explanations grouped into each context with associated training labels and SCNN predictions a) normal, b) hyperplastic, c) adenomatous and d) cancerous.

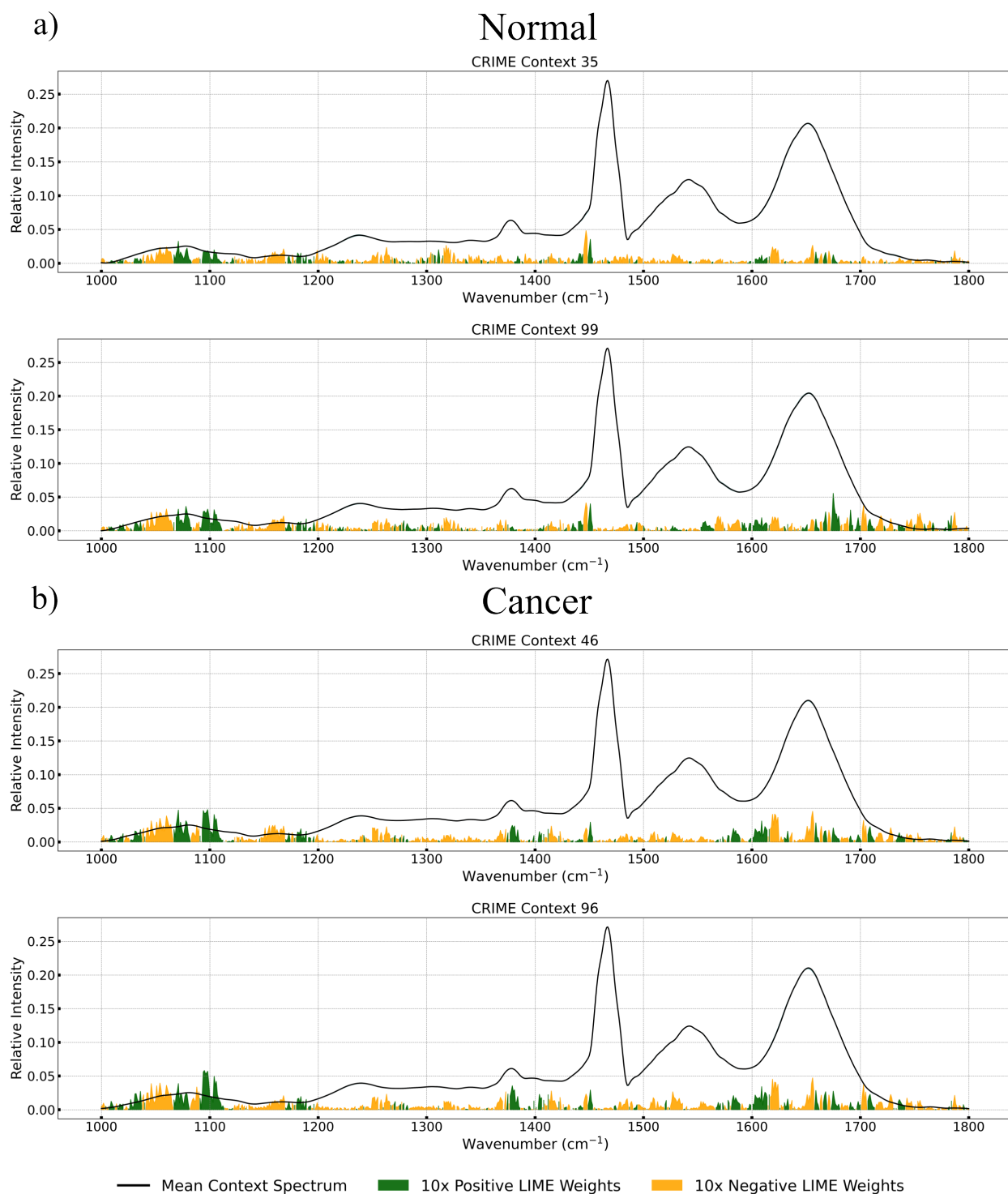


Figure 20: CRIME context average spectra and LIME weightings demonstrating the ability of the CRIME method to

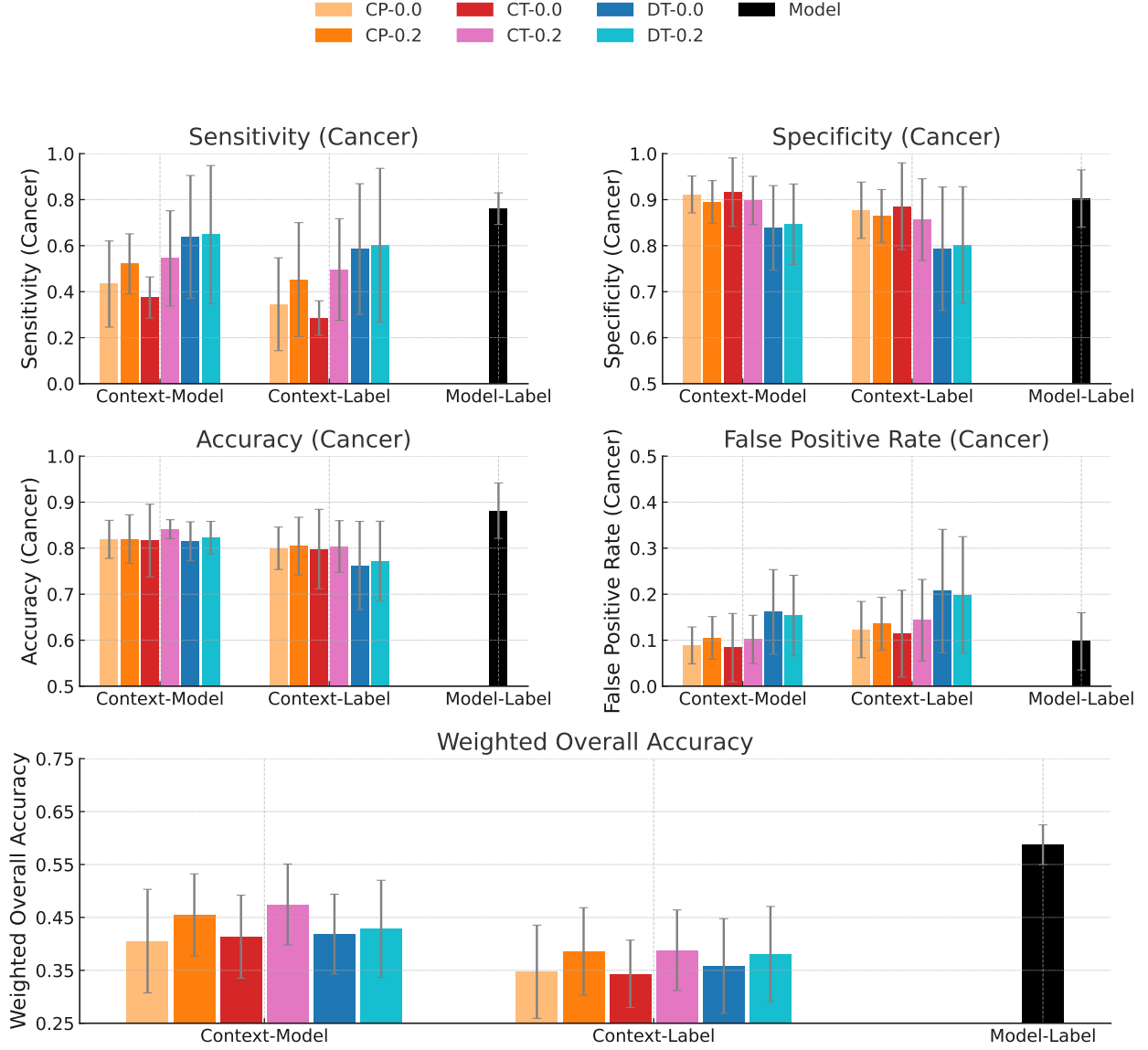


Figure 21: LCRIME four-class classification performance (mean $\pm$ std) across folds with $K = 100$ clustering. The VAE model used is denoted using the convention outlined in Table ??, followed by the decimalised value of the supervision level used.

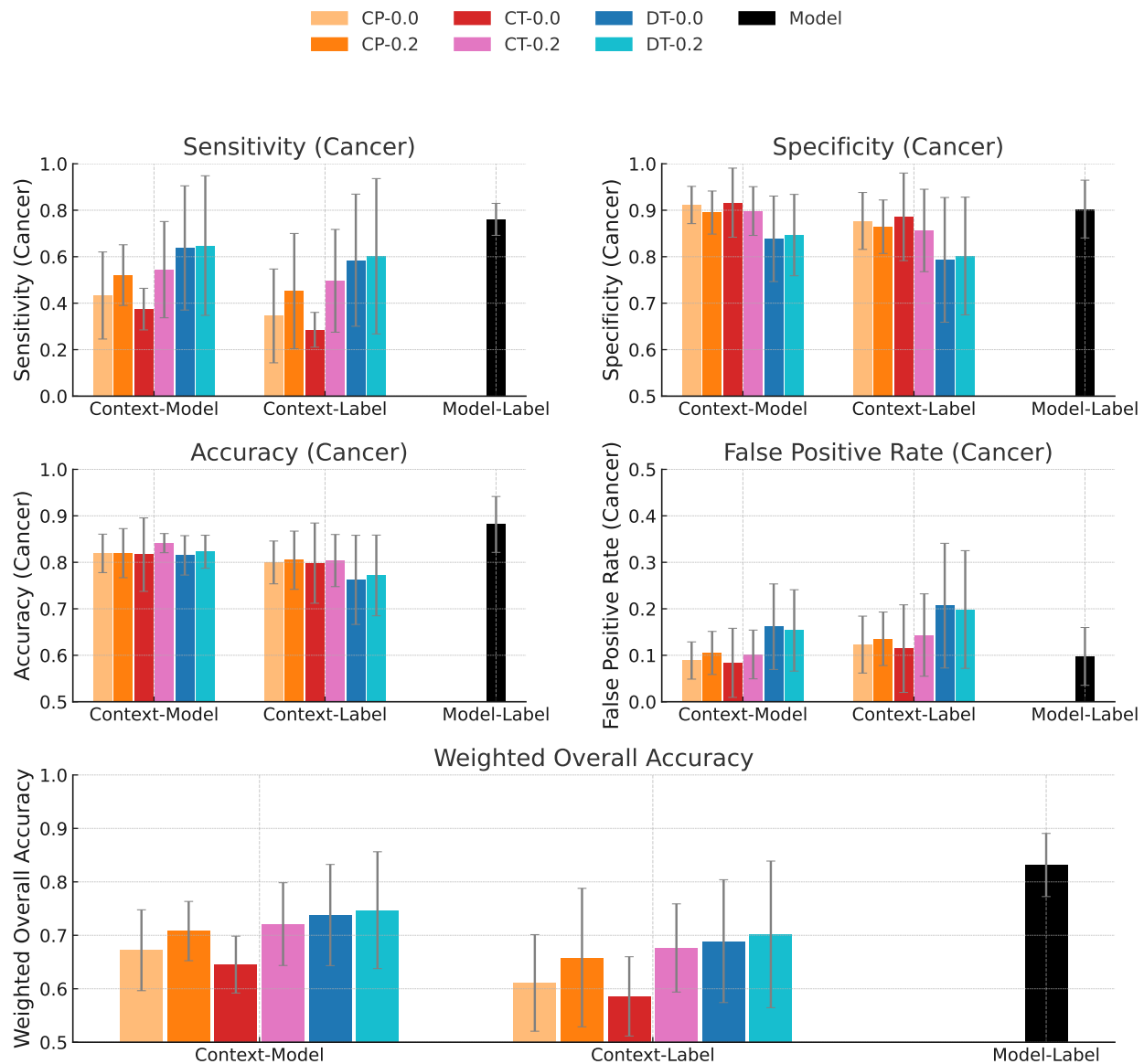


Figure 22: LCRIME binary classification performance (mean $\pm$ std) across folds with $K = 100$ clustering. The VAE model used is denoted using the convention outlined in Table ??, followed by the decimalised value of the supervision level used.

3.6 Applicability in Physics

The use of machine learning and deep learning methods are becoming increasingly common in physics. Similar approaches to the VAE method adopted here have been used to detect and reconstruct collisions in particle physics [[cheng'variational'2023](#), [cerri'variational'2019](#), [alves'variational'2024](#)], perform discovery and age-prediction tasks in astrophysical contexts [[andika'accelerating'2025](#), [leung'variational'2023](#), [gagliano'physics-informed'2023](#)], detect gravitational waves [[gabbard'bayesian'2022](#), [bayley'rapid'2022](#), [bayley'robust'2020](#), [yamamoto'use'2020](#)], and perform cancer diagnosis and severity classification tasks in medical physics, as demonstrated and exemplified throughout this work. Increasing the interpretability of these methods such as has been done in this work, is therefore important for translating new-found results into real physical insights.

4 Conclusions

In this work, I have explored the use of MSE, cosine similarity, PCA informed LDA, PCA informed XGBoost, standard XGB and ensemble SCNN methods for the classification of colon cancer samples as non-cancerous, hyperplastic, adenomatous, or cancerous via the analysis of FTIR spectral data. The SCNN method was found to be the most effective, achieving an OWA of $58.7 \pm 3.7\%$ when performing four-class classification of on a four-fold cross-validated test set. When mapped to binary classification, the SCNN also performs best and achieves non-cancer vs cancer classification with OWA $83.2 \pm 5.9\%$ - similar to the accuracy achieved by Lasalvia et al. [[lasalvia'discrimination'2023](#)]. This model was then explained using LIME and CRIME techniques to extract multiple contexts to explain each tissue classification. The spectra and weightings associated with these contexts were also used to create LCRIME models. The best of these LCRIME models for 4-class classification utilised a 20% supervised CP-VAE. This model was capable of reproducing SCNN predictions with OWA $47.5 \pm 7.6\%$ on the four-fold cross-validated test set. For binary classification, the best LCRIME model utilised a 20% supervised DT-VAE and achieved an OWA of $74.7 \pm 10.9\%$ on the same test set.

Future works could improve the performance of the classification models by implementing ordinal regressive losses [[empty citation](#)] during SCNN training, implementing transformer networks directly for classification, or exploring the use of different weak learner ensemble methods instead of simple voting [[empty citation](#)]. Furthermore, the explainability work could be expanded on by utilising different dataset folds to achieve finer hyperparameter tuning for the CRIME VAEs, and by exploring the use of other explainability methods like SHAP [[empty citation](#)].

A Standard Machine Learning Methods

A.1 Principal Component Analysis (PCA)

PCA [wold'principal'nodate] is a dimensionality reduction technique which aims to project high-dimensional data into a new, more meaningful basis by re-expressing the data through a linear combination of these new basis vectors. This linearity allows us to pose the problem through linear algebra, where we wish to find a linear transformation $\mathbf{P}$ which maps the original recorded dataset $\mathbf{X}$ to a new representation of that dataset $\mathbf{Y}$ such that:

$$\mathbf{Y} = \mathbf{P} \cdot \mathbf{X}. \quad (17)$$

The rows of $\mathbf{P}$ then form a new set of orthonormal basis vectors. To find $\mathbf{P}$, we assume that axes of interest are those with high variance. These high variance axes can be found by calculating the eigenvectors of the covariance matrix of the dataset:

$$\mathbf{C} = \frac{1}{n} \mathbf{X} \mathbf{X}^T, \quad (18)$$

which itself gives the degree of linear relationship between different variables. The eigenvalues of $\mathbf{C}$ therefore correspond to the variance of $\mathbf{X}$ along the associated eigenvector [shlens'tutorial'2014]. Throughout this work, PCA is used to reduce inputs to 25 dimensions.

A.2 Linear Discriminant Analysis (LDA)

LDA [fisher'use'1936] is a classification technique which aims to maximise the separability of classifications for given input data (e.g. IR spectra) by maximising between-class variance and minimising within-class variance [tharwat'linear'2017]. This is achieved by assuming data corresponding to a specific class k is normally distributed about a mean value μ_k , and that all classes have equal covariance matrices $\Sigma_k = \Sigma$. The first assumption requires there to be a point x^* which is equally likely to belong to two classes. If we take x^* as the decision boundary, we can then minimise the classification error, which in the binary classification case with classes $\mathcal{C}_1$ and $\mathcal{C}_2$ is given by:

$$\mathbb{P}(\text{error}) = \mathbb{P}(x > x^*, x \in \mathcal{C}_1) + \mathbb{P}(x < x^*, x \in \mathcal{C}_2). \quad (19)$$

Minimising this error with respect to x^* , denoting the probability density within each class as $f_k(x)$ and denoting priors for each class by π_k then gives;

$$f_1(x^*)\pi_1 = f_2(x^*)\pi_2. \quad (20)$$

Now considering multi-class classification and assuming the data follows a multivariate Gaussian distribution of dimension d , we have;

$$f_k(\mathbf{x})\pi_k = \frac{1}{\sqrt{(2\pi)^d |\Sigma_k|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x} - \mu_k)\right) \pi_k. \quad (21)$$

Taking the logarithm of this, discounting constant terms and enforcing the equal covariance assumption gives scaled posterior probabilities for each class k as:

$$\delta_k(\mathbf{x}) = \mu_k^T \Sigma^{-1} \mathbf{x} - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \ln(\pi_k). \quad (22)$$

The classification k with maximal $\delta_k(\mathbf{x})$ is then chosen as the predicted class for $\mathbf{x}$ [ghojogh'linear'2019]. In practice, the priors π_k are estimated according to the proportion of samples in each class, and the means μ_k and covariance matrices Σ_k are estimated using maximum likelihood estimation (MLE) [haynes'maximum'2013]. In the mean case, this is given by:

$$\mu_k = \frac{1}{n_k} \sum_{i=1}^n \mathbf{x}_i \mathbb{I}(\mathcal{C}(\mathbf{x}_i) = k), \quad (23)$$

where $\mathbb{I}$ is the indicator function [ghojogh'linear'2019]. As we assume the covariance matrices are the same for each class, we take Σ to be the weighted sum of the estimated covariance matrices for each class. This can be considered geometrically [mika'fisher'1999, tharwat'linear'2017] by solving an eigenvalue problem as is shown in Figure ???. New data can then be classified by finding the projected training class mean which minimises the distance to the projection of test data within the eigenvector space.

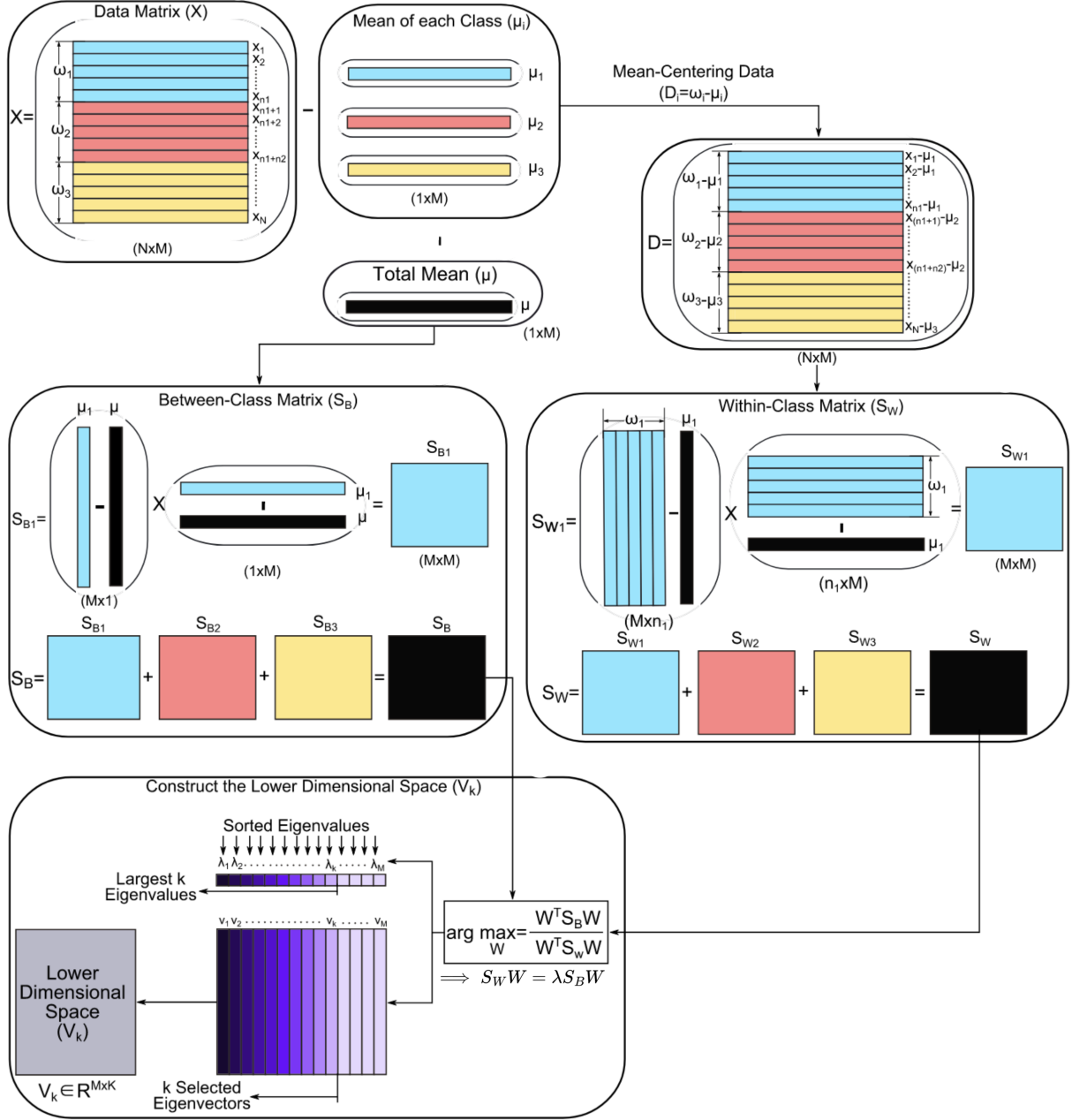


Figure 23: A graphical representation of the LDA algorithm. Each class ω_k is coloured differently. Note the use of Fisher's criterion in the lowest cell which is rewritten in eigen-equation form; $S_W W = \lambda S_B W$. Modified from [tharwat'linear'2017].

A.3 Extreme Gradient Boosting (XGBoost or XGB)

XGBoost generates an ensemble of regression T-trees (trees with T leaves) - producing K additive functions f_k , which combined, learn to map inputs to continuous values

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad (24)$$

subject to the regularisation objective

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k). \quad (25)$$

Here, l is a differentiable loss between the prediction $\hat{y}_i$ and target value y_i , and $\Omega(f)$ is a complexity penalising regularisation term [chen'xgboost'2016]. We assemble the ensemble of trees iteratively. At each iteration t , we add f_t to the prediction at the previous iteration and minimise

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (26)$$

$$\approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t), \quad (27)$$

where $g_i = \partial_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$ and $h_i = \partial_{\hat{y}_i^{(t-1)}}^2 l(y_i, \hat{y}_i^{(t-1)})$ are first and second order gradients of the loss function [friedman'additive'2000]. Minimising this loss over a leaf then gives the optimal tree weighting w_j for that leaf. Many further optimisation techniques are used to improve the performance of XGBoost, the details of which can be found in [chen'xgboost'2016]. A final prediction is then made by summing contributions from each tree for a given input, as shown in Figure ?? . Note that for N -classification tasks, N trees - corresponding to positive and negative classification for each class - are built at each iteration, with softmax [bridle'probabilistic'1990] over the summed outputs from these trees used to select the best classification.

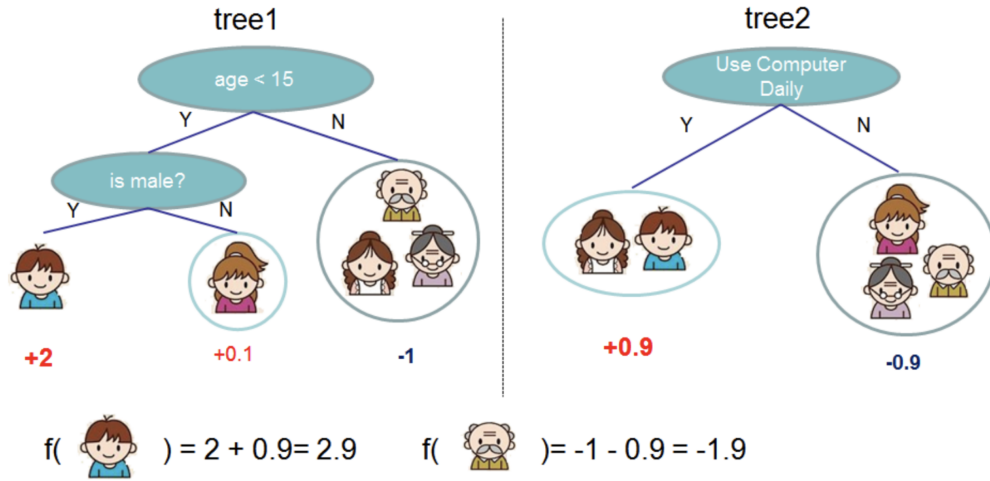


Figure 24: A tree ensemble model, where final predictions are the sum of those from each tree. From [chen'xgboost'2016].

B ML Connection Layers

There are many ways to connect the layers of a neural network, of which three are used throughout this work; fully connected layers, convolutional layers and transformer layers.

Fully Connected Layers [minsky'perceptrons'2017, rosenblatt'principles'nodate] are those where there is a connection from any node in one layer to all nodes in the subsequent layer, with each connection having an associated weight and bias such that the output in a subsequent layer is given by:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad (28)$$

where $\mathbf{W}$ is a weight matrix and $\mathbf{b}$ is a bias vector [rosenblatt'principles'nodate]. By applying many fully connected layers together and sending the output value after each layer through a non-linear activation function, we form a multi-layer perception (MLP) which is able to learn any function [cybenko'approximation'1989, leshno'multilayer'1993, hornik'multilayer'1989]. This fully-connected approach is illustrated in Figure ??a.

Convolutional Layers [kiranyaz'1d'2019, lecun'gradient-based'1998, goodfellow'deep'2016] have a set of learnable kernels each of the same size which are slid across an input tensor. At each position along this slide, the kernel is convolved with the respective elements of the input tensor, producing one value in the next layer. Using multiple kernels for this allows for different features to be extracted [dumoulin'guide'2018]. Convolutional layers also have two tunable parameters. Stride determines how many times the kernel should slide over the input tensor between convolution calculations, decreasing the number of calculated values. Padding determines the number of zero-valued layers to surround the input with, which can be used to allow every real data value to be processed by every element of the kernel. The convolutional approach is illustrated in Figure ??b.

Transformer Layers first produce three latent space embeddings from input data, normally labelled as the query, $\mathbf{Q}$, key, $\mathbf{K}$ and value $\mathbf{V}$. The query and key are then used to calculate attention weights $\mathbf{A}$ according to:

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right), \quad (29)$$

where d_k is the dimensionality of the key [vaswani'attention'2023]. These attention weights are then used to multiply the value embedding vector to produce intermediate values $\mathbf{I}$;

$$\mathbf{I} = \mathbf{A}\mathbf{V}. \quad (30)$$

This 'scaled dot-product attention' process encodes long-range relationships between components of the input vector. If this process is repeated h times, then we can produce h attention heads and h corresponding intermediate values $\mathbf{I}_{0...h}$. To then enable the learning of positioning information, these output head values are concatenated and passed through a linear layer to form a vector of the same shape as the input. A positional encoding, normally a scaled $\sin(\text{index}/\text{input vector length})$ function, is then added to this vector to give a positionally encoded attention vector $\mathbf{j}$. Finally, to allow for mixing of the different head encodings, this vector is fed through a two-layer fully-connected feed-forward network (FFN) with layer weights $\mathbf{W}_i$ and biases $\mathbf{b}_i$. Each layer is operated on by ReLU activations, enabling the learning of arbitrarily complex functions [vaswani'attention'2023]. To retain positional information $\mathbf{j}$ is then added to this representation, giving the overall output from the transformer $\mathbf{O}$ as;

$$\mathbf{O} = \text{FNN}(\mathbf{j}) = \text{ReLU}(\mathbf{W}_1\mathbf{j} + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 + \mathbf{j}. \quad (31)$$

In practice, we also perform layer normalisation after the multi-head attention and FNN steps. This process is illustrated in Figure ??c.

C Variational Autoencoder Derivation

An unsupervised VAE must learn a joint distribution $p_\theta(\mathbf{x}, \mathbf{z})$ such as to maximise the marginal likelihood of the data:

$$p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z} \quad (32)$$

$$= \int_{\mathbf{z}} p_\theta(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z}) d\mathbf{z} \quad (33)$$

Equation (??), however is normally intractable. To overcome this, we use Bayes' theorem to rewrite the marginal likelihood as:

$$p_\theta(\mathbf{x}) = \frac{p_\theta(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z})}{p_\theta(\mathbf{z}|\mathbf{x})} = \frac{p_\theta(\mathbf{x}, \mathbf{z})}{p_\theta(\mathbf{z}|\mathbf{x})}. \quad (34)$$

Therefore, we can also approach the problem by attempting to approximate the posterior $p(\mathbf{z}|\mathbf{x})$ by an amortised variational distribution $q_\phi(\mathbf{z}|\mathbf{x})$ with parameters ϕ . By then expanding the Kullback-Leibler (KL) divergence [**<empty citation>**]:

$$D_{\text{KL}}(q|p) = \int q(\mathbf{z}) \log \left(\frac{q(\mathbf{z})}{p(\mathbf{z})} \right) d\mathbf{z} \geq 0, \quad (35)$$

which measures the information entropy of q_ϕ relative to p_θ , we can write the marginal likelihood as:

$$\log p_\theta(\mathbf{x}) = D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})|p_\theta(\mathbf{z}|\mathbf{x})) + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{z}, \mathbf{x}) - \log q_\phi(\mathbf{z}|\mathbf{x})] \quad (36)$$

$$\implies \log p_\theta(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{z}, \mathbf{x}) - \log q_\phi(\mathbf{z}|\mathbf{x})] = \text{ELBO}, \quad (37)$$

where the term on the RHS of the inequality is the evidence lower bound (ELBO) [**<empty citation>**]. The ELBO gives a lower bound on the marginal likelihood, so maximising it also minimises the KL divergence. This maximisation, however is non-trivial because the ϕ derivative of the ELBO is intractable and non-estimable. To overcome this, we employ the reparameterisation trick [**<empty citation>**], where we deterministically re-express $\mathbf{z}$ as:

$$\mathbf{z} = g(\phi, \mathbf{x}, \epsilon), \quad (38)$$

where ϵ is a random variable generated through a Gaussian distribution with mean $\mathbf{0}$ and unit variance [**SGVB'gradient'variational'bayes**]. Finally, we can estimate the gradient of the negative $-\text{ELBO} = \mathcal{L}(\mathbf{x})$ (to perform gradient descent) as:

$$\mathcal{L}(\mathbf{x}) = -\mathbb{E}_{p(\epsilon)} [\log p_\theta(\mathbf{z}, \mathbf{x}) - \log q_\phi(\mathbf{z}|\mathbf{x})] \quad (39)$$

$$\implies \nabla_{\theta, \phi} \mathcal{L}(\mathbf{x}) = -\frac{1}{N} \sum_{i=1}^N \nabla_{\theta, \phi} [\log p_\theta(\mathbf{z}, \mathbf{x}) - \log q_\phi(\mathbf{z}|\mathbf{x})], \quad (40)$$

where N is the number of samples in a batch. This formulation allows us to approximate the posterior by gradient descent. The ELBO can now be expanded to give:

$$\text{ELBO} = \mathbb{E}_{p(\epsilon)} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})|p_\theta(\mathbf{z})). \quad (41)$$

If we then assume the prior $p_\theta(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, and that the distribution $q_\phi(\mathbf{z}|\mathbf{x})$ is a multivariate Gaussian with mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma}$ where $\boldsymbol{\mu}, \boldsymbol{\sigma} = f(\mathbf{x})$, we can rewrite the KL divergence ?? as:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})|p_\theta(\mathbf{z})) = -\frac{1}{2} (1 + \log(\boldsymbol{\sigma}^2) - \boldsymbol{\mu}^2 - \boldsymbol{\sigma}^2). \quad (42)$$

Furthermore, we use single-sample monte carlo estimation [**<empty citation>**] to find the expectation of the log-likelihood term subject to the assumption that $p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{y}_\mu, 1)$, which reduces to the form:

$$\log p_\theta(\mathbf{x}|\mathbf{z}) = -\frac{1}{2} (\mathbf{x} - \mathbf{y}_\mu)^2 = -\frac{1}{2} \text{MSE}(\mathbf{x}, \mathbf{y}_\mu). \quad (43)$$

Hence, we can optimise such a VAE by minimising the negative ELBO:

$$\mathcal{L}(\mathbf{x}) = -\frac{1}{2} \text{MSE}(\mathbf{x}, \mathbf{y}_\mu) + \beta \frac{1}{2} [1 + \log(\boldsymbol{\sigma}^2) - \boldsymbol{\mu}^2 - \boldsymbol{\sigma}^2], \quad (44)$$

which is adopted as the unsupervised VAE loss function throughout this work, where β is a tunable parameter [**beta-vae**].

D Activation Functions

ReLU (Rectified Linear Unit), Leaky ReLU (with gradient parameter α) and tanh activation functions are depicted in Figure ???. They can also be expressed mathematically as:

$$\begin{aligned}\text{ReLU}(x) &= \max(0, x) \\ \text{Leaky ReLU}(x) &= \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases} \\ \tanh(x) &= \frac{e^x - e^{-x}}{e^x + e^{-x}}\end{aligned}$$

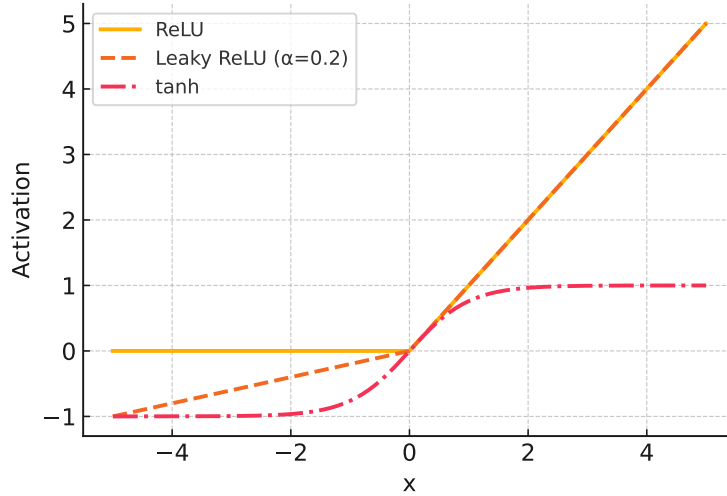


Figure 25: Comparison of ReLU, Leaky ReLU, and tanh activation functions.

E Traditional ML Classification Performance With 10-Fold Cross-Validation

Model	Metric	Normal	Hyperplastic	Adenoma	Cancer	Accuracy
PCA > LDA	Sensitivity	42 ± 20.4	31 ± 17.7	84 ± 14.3	39 ± 26.3	61 ± 7.6
	Specificity	90 ± 11.1	94 ± 5.4	57 ± 19.9	95 ± 6.2	—
XGBoost	Sensitivity	58 ± 11.9	41 ± 14.7	80 ± 14.4	62 ± 33.6	68 ± 10.0
	Specificity	88 ± 6.6	91 ± 2.6	77 ± 10.2	96 ± 6.2	—
PCA > XGBoost	Sensitivity	43 ± 11.5	28 ± 11.9	55 ± 11.9	75 ± 20.9	51 ± 9.2
	Specificity	88 ± 10.3	90 ± 6.7	79 ± 7.7	77 ± 10.5	—

Table 6: Performance characteristics (mean ± SD) for PCA fed LDA, PCA fed XGBoost and raw data into XGBoost methods for four-classification prediction with 10-fold cross-validation as found by Azevedo (unpublished).

F Relevant Code

F.1 Scale-Adjusting Convolutional Neural Network

```
1 class TPL(nn.Module):
2     def __init__(self):
3         super(TPL, self).__init__()
4
5     def forward(self, a, b, c, x):
6         Px = c+(1-c)/(1+torch.exp(-a*(x-b)))
7         return Px
8
9 class SCNN_TPL3(nn.Module):
10     name = "SCNN_TPL3"
11
12     def __init__(self, apriori_relevance, leak_rate, dropout_rate, input_channels):
13         super(SCNN_TPL3, self).__init__()
14
15         # Multi-scale Assessing Layers
16         self.ms18 = nn.Conv1d(in_channels=1, out_channels=8, kernel_size=8, stride=4)
17         self.ms1_norm_8 = nn.BatchNorm1d(8)
18         self.ms125 = nn.Conv1d(in_channels=1, out_channels=8, kernel_size=25, stride=12)
19         self.ms1_norm_25 = nn.BatchNorm1d(8)
20         self.ms150 = nn.Conv1d(in_channels=1, out_channels=8, kernel_size=50, stride=25)
21         self.ms1_norm_50 = nn.BatchNorm1d(8)
22         self.ms1_norm_801 = nn.BatchNorm1d(1)
23
24         # An array of length 801 denoting an apriori understanding of the importance of each channel
25         self.apriori_relevance = apriori_relevance
26
27         # Core CNN Layers
28         self.relu = nn.LeakyReLU(leak_rate)
29         self.tanh = nn.Tanh()
30         self.dropout = nn.Dropout(p=dropout_rate)
31
32         self.core1 = nn.Conv1d(in_channels=input_channels, out_channels=8, kernel_size=15, stride=7)
33         self.core_norm1 = nn.BatchNorm1d(8)
34
35         self.core2_1 = nn.Conv1d(in_channels=8, out_channels=16, kernel_size=9, stride=4)
36         self.core_norm2_1 = nn.BatchNorm1d(16)
37
38         self.core3_1 = nn.Conv1d(in_channels=16, out_channels=32, kernel_size=9, stride=4)
39         self.core_norm3_1 = nn.BatchNorm1d(32)
40
41         self.core4 = nn.Conv1d(in_channels=32, out_channels=64, kernel_size=2)
42         self.core_norm4 = nn.BatchNorm1d(64)
43
44         self.core5 = nn.Linear(1600, 32)
45         self.core_norm5 = nn.BatchNorm1d(32)
46
47         self.core6 = nn.Linear(32,16)
48         self.core_norm6 = nn.BatchNorm1d(16)
49
50         ### Output Layers
51         # Determine a,b,c parameters for TPL
52         self.outa = nn.Linear(16,4)
53         self.outb = nn.Linear(16,4)
54         self.outc = nn.Linear(16,4)
55         self.outx = nn.Linear(16,4)
56
57         # Determine output values for each class
58         self.tpl = TPL()
```

```

59
60 def forward(self, x):
61
62     # Multi-Scale assessing Layer
63     x_ms8 = self.ms8(x)
64     x_ms8 = self.ms8_norm(x_ms8)
65     x_ms25 = self.ms25(x)
66     x_ms25 = self.ms25_norm(x_ms25)
67     x_ms50 = self.ms50(x)
68     x_ms50 = self.ms50_norm(x_ms50)
69     x_ms_apriori = x * self.apriori_relevance
70     x_ms_apriori = self.ms801_norm(x_ms_apriori)
71
72     x_ms_convs = torch.concat([x_ms8, x_ms25, x_ms50], axis=2)
73     x_ms_convs_flat = x_ms_convs.view(x_ms_convs.size(0), -1)
74     x_ms_apriori_flat = x_ms_apriori.view(x_ms_apriori.size(0), -1)
75     x_core = torch.concat([x_ms_convs_flat, x_ms_apriori_flat], axis=1)
76     x_core = x_core.view(len(x_core), 1, x_core.size(1))
77
78     # Core CNN Layers
79     # First Convolution Layer
80     x_core = self.core1(x_core)
81     x_core = self.core_norm1(x_core)
82     x_core = self.dropout(x_core)
83     x_core = self.relu(x_core)
84
85     # First tanh-relu block
86     x_core = self.core2_1(x_core)
87     x_core = self.core_norm2_1(x_core)
88     x_core = self.dropout(x_core)
89     x_core = self.tanh(x_core)
90     x_core = self.relu(x_core)
91
92     # Second tanh-relu block
93     x_core = self.core3_1(x_core)
94     x_core = self.core_norm3_1(x_core)
95     x_core = self.dropout(x_core)
96     x_core = self.tanh(x_core)
97     x_core = self.relu(x_core)
98
99     # Fine convolution layer
100    x_core = self.core4(x_core)
101    x_core = self.core_norm4(x_core)
102    x_core = self.dropout(x_core)
103    x_core = x_core.view(len(x_core), -1)
104
105    ## Linear Layers
106    x_core = self.core5(x_core)
107    x_core = self.core_norm5(x_core)
108    x_core = self.dropout(x_core)
109    x_core = self.tanh(x_core)
110    x_core = self.relu(x_core)
111
112    x_core = self.core6(x_core)
113    x_core = self.core_norm6(x_core)
114    x_core = self.dropout(x_core)
115    x_core = self.tanh(x_core)
116    x_core = self.relu(x_core)
117
118    # Output Layers
119    a = self.outa(x_core)
120    b = self.outb(x_core)
121    c = self.outc(x_core)

```

```

122         x_out = self.outx(x_core)
123         x_out = self.tpl(a,b,c,x_out)
124
125         return x_out

```

F.2 VAE Loss Functions

```

1  # Incorporate the Kullback-Leibler loss prevents the divergence of probability distributions
2  def kl_mse_loss(recon_x, x, mu, logvar, beta=1):
3      mse = nn.functional.mse_loss(recon_x, x, reduction='sum') / x.size(0)
4      kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0) * beta
5      return mse + kl, mse, kl
6
7  def elbo_supervised(x, x_rec, mu, logvar, y_logits, y_true, beta=1, label_smoothing=0.25):
8      mse = nn.functional.mse_loss(x_rec, x, reduction='sum') / x.size(0)
9      kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0)
10     ce = nn.functional.cross_entropy(y_logits, y_true, label_smoothing=label_smoothing, reduction='sum') / x.size(0)
11     return mse + kl*beta, mse, kl, ce
12
13  def elbo_unsupervised(x, model, n_classes=4, beta=1):
14      logits, q_y, mu, logvar, hidden = model.infer(x)
15      elbo_y = []
16      for k in range(n_classes):
17          y_k = F.one_hot(torch.full((x.size(0),), k, dtype=torch.long, device=x.device), n_classes).float()
18          # recompute mu/logvar conditioned on y_k
19          mu_k = model.fc_mu(torch.cat([hidden, y_k], dim=1))
20          logvar_k = model.fc_logvar(torch.cat([hidden, y_k], dim=1))
21          z_k = reparameterise(mu_k, logvar_k, eps_scale=model.eps_scale)
22          # reconstruction term
23          x_rec_k = model.decoder(torch.cat([z_k, y_k], dim=1))
24          mse_k = -F.mse_loss(x_rec_k, x, reduction='none').sum(dim=1)
25          # KL term
26          kl_k = -0.5 * torch.sum(1 + logvar_k - mu_k.pow(2) - logvar_k.exp(), dim=1) * beta
27          elbo_y.append(mse_k - kl_k)
28      elbo_tensor = torch.stack(elbo_y, dim=1)
29      # weighted sum and entropy
30      expected = torch.sum(q_y * elbo_tensor, dim=1)
31      entropy = -torch.sum(q_y * torch.log(q_y + 1e-8), dim=1)
32      return torch.mean(expected + entropy)

```

F.3 Reparameterisation Trick

```

1  def reparameterise(mu, logvar, eps_scale=1):
2      std = torch.exp(0.5 * logvar)
3      eps = torch.randn_like(std) * eps_scale
4      return mu + eps * std

```

G Full Classification Results

The full results for performance in four-class and binary classification tasks are presented below. Tables ?? and ?? show the results when using traditional ML methods and when using the ensemble SCNN method developed in this work. Tables ?? and ?? show the results when using difference VAE architectures to generate context-based classification models. All results are generated using four-fold cross-validation as described in §??. Within these latter two tables, the method denotes which VAE was used to produce the context-based classification model. CP denotes a convolutional encoder and MLP decoder; CT denotes a convolutional encoder and transformer decoder; and DT denotes a transformer encoder and transformer decoder. The value after the two following digits denotes the supervision level; 0.2 $\implies$ 20% supervision. The comparison column denotes which outputs are compared. For example Context-Model compares the

Context classification predictions to the SCNN Model predictions and Context-Label compares the context classification predictions to the original dataset labels. In each case, we assume the latter metric is truthful. The SCNN Model-Label predictions are included in the final row of both tables for comparison.

Method	Metric	Normal	Hyperplastic	Adenoma	Cancer
MSE	Sensitivity	0.238 ± 0.032	0.396 ± 0.138	0.310 ± 0.056	0.829 ± 0.114
	Specificity	0.853 ± 0.040	0.823 ± 0.098	0.812 ± 0.088	0.703 ± 0.067
	Accuracy	0.733 ± 0.030	0.755 ± 0.073	0.562 ± 0.047	0.722 ± 0.068
	False Positive Rate	0.147 ± 0.040	0.177 ± 0.098	0.188 ± 0.088	0.297 ± 0.067
	OWA	0.443 ± 0.062			
Cosine Similarity	Sensitivity	0.238 ± 0.032	0.396 ± 0.138	0.310 ± 0.056	0.829 ± 0.114
	Specificity	0.853 ± 0.040	0.823 ± 0.098	0.812 ± 0.088	0.703 ± 0.067
	Accuracy	0.733 ± 0.030	0.755 ± 0.073	0.562 ± 0.047	0.722 ± 0.068
	False Positive Rate	0.147 ± 0.040	0.177 ± 0.098	0.188 ± 0.088	0.297 ± 0.067
	OWA	0.443 ± 0.062			
PCA $\rightarrow$ LDA	Sensitivity	0.430 ± 0.043	0.230 ± 0.099	0.800 ± 0.075	0.440 ± 0.294
	Specificity	0.890 ± 0.062	0.950 ± 0.041	0.550 ± 0.174	0.930 ± 0.042
	Accuracy	0.800 ± 0.052	0.830 ± 0.044	0.670 ± 0.090	0.860 ± 0.071
	False Positive Rate	0.110 ± 0.062	0.050 ± 0.041	0.450 ± 0.174	0.070 ± 0.042
	OWA	0.476 ± 0.079			
PCA $\rightarrow$ XGB	Sensitivity	0.470 ± 0.096	0.350 ± 0.090	0.770 ± 0.051	0.590 ± 0.075
	Specificity	0.880 ± 0.030	0.900 ± 0.047	0.710 ± 0.094	0.940 ± 0.054
	Accuracy	0.800 ± 0.028	0.810 ± 0.034	0.740 ± 0.066	0.880 ± 0.050
	False Positive Rate	0.120 ± 0.030	0.100 ± 0.047	0.290 ± 0.094	0.060 ± 0.054
	OWA	0.544 ± 0.053			
XGBoost	Sensitivity	0.50 ± 0.093	0.38 ± 0.115	0.78 ± 0.048	0.61 ± 0.079
	Specificity	0.89 ± 0.025	0.91 ± 0.031	0.71 ± 0.089	0.94 ± 0.048
	Accuracy	0.81 ± 0.022	0.83 ± 0.009	0.75 ± 0.064	0.89 ± 0.048
	False Positive Rate	0.11 ± 0.025	0.09 ± 0.031	0.29 ± 0.089	0.06 ± 0.048
	OWA	0.568 ± 0.058			
Ensemble SCNN	Sensitivity	0.707 ± 0.102	0.556 ± 0.068	0.325 ± 0.054	0.761 ± 0.069
	Specificity	0.778 ± 0.082	0.868 ± 0.084	0.887 ± 0.080	0.903 ± 0.062
	Accuracy	0.765 ± 0.058	0.714 ± 0.025	0.796 ± 0.072	0.882 ± 0.060
	False Positive Rate	0.222 ± 0.082	0.132 ± 0.084	0.113 ± 0.080	0.098 ± 0.062
	OWA	0.587 ± 0.037			

Table 7: Four-class classification performance (mean $\pm$ std) across folds for MSE, Cosine Similarity, PCA $\rightarrow$ LDA, PCA $\rightarrow$ XGBoost, XGBoost, and SCNN methods. The MSE and Cosine Similarity rows repeat identical values, as performances were the same to three significant figures. OWA = Overall Weighted Accuracy.

Method	Metric	Non-Cancer	Cancer
MSE	Sensitivity	0.703 ± 0.067	0.829 ± 0.114
	Specificity	0.829 ± 0.114	0.703 ± 0.067
	Accuracy	0.722 ± 0.068	0.722 ± 0.068
	False Positive Rate	0.171 ± 0.114	0.297 ± 0.067
	Overall Weighted Accuracy	0.766 ± 0.080	
Cosine Similarity	Sensitivity	0.703 ± 0.067	0.829 ± 0.114
	Specificity	0.829 ± 0.114	0.703 ± 0.067
	Accuracy	0.722 ± 0.068	0.722 ± 0.068
	False Positive Rate	0.171 ± 0.114	0.297 ± 0.067
	Overall Weighted Accuracy	0.766 ± 0.080	
PCA $\rightarrow$ LDA	Sensitivity	0.93 ± 0.042	0.44 ± 0.294
	Specificity	0.44 ± 0.294	0.93 ± 0.042
	Accuracy	0.86 ± 0.071	0.86 ± 0.071
	False Positive Rate	0.56 ± 0.294	0.07 ± 0.042
	Overall Weighted Accuracy	0.687 ± 0.162	
PCA $\rightarrow$ XGB	Sensitivity	0.93 ± 0.055	0.60 ± 0.075
	Specificity	0.60 ± 0.075	0.93 ± 0.055
	Accuracy	0.89 ± 0.049	0.89 ± 0.049
	False Positive Rate	0.40 ± 0.075	0.07 ± 0.055
	Overall Weighted Accuracy	0.769 ± 0.049	
XGB	Sensitivity	0.94 ± 0.047	0.61 ± 0.084
	Specificity	0.61 ± 0.084	0.94 ± 0.047
	Accuracy	0.89 ± 0.049	0.89 ± 0.049
	False Positive Rate	0.39 ± 0.084	0.06 ± 0.047
	Overall Weighted Accuracy	0.773 ± 0.060	
Ensemble SCNN	Sensitivity	0.903 ± 0.062	0.761 ± 0.069
	Specificity	0.761 ± 0.069	0.903 ± 0.062
	Accuracy	0.882 ± 0.060	0.882 ± 0.060
	False Positive Rate	0.239 ± 0.069	0.098 ± 0.062
	Overall Weighted Accuracy	0.832 ± 0.059	

Table 8: Binary classification performance (mean $\pm$ std) across folds for MSE, Cosine Similarity, PCA $\rightarrow$ LDA, PCA $\rightarrow$ XGB, XGB, and SCNN methods. The MSE and Cosine Similarity rows repeat identical values, as performances were the same to three significant figures.

Method	Comparison	Metric	Non-c.	Hyper.	Adenoma	Cancer
CP-0.0	Context-Model	Sensitivity	0.531 $\pm$ 0.316	0.338 $\pm$ 0.136	0.319 $\pm$ 0.192	0.433 $\pm$ 0.187
		Specificity	0.674 $\pm$ 0.255	0.847 $\pm$ 0.023	0.786 $\pm$ 0.227	0.911 $\pm$ 0.040
		Accuracy	0.662 $\pm$ 0.111	0.680 $\pm$ 0.049	0.697 $\pm$ 0.165	0.819 $\pm$ 0.041
		False Positive Rate	0.326 $\pm$ 0.255	0.153 $\pm$ 0.023	0.214 $\pm$ 0.227	0.089 $\pm$ 0.040
		Overall Weighted Acc.		0.405 $\pm$ 0.098		
	Context-Label	Sensitivity	0.494 $\pm$ 0.280	0.262 $\pm$ 0.059	0.290 $\pm$ 0.205	0.345 $\pm$ 0.202
		Specificity	0.625 $\pm$ 0.275	0.821 $\pm$ 0.067	0.786 $\pm$ 0.223	0.877 $\pm$ 0.061
		Accuracy	0.602 $\pm$ 0.177	0.544 $\pm$ 0.033	0.706 $\pm$ 0.157	0.800 $\pm$ 0.046
		False Positive Rate	0.375 $\pm$ 0.275	0.178 $\pm$ 0.067	0.215 $\pm$ 0.223	0.123 $\pm$ 0.202
		Overall Weighted Acc.		0.347 $\pm$ 0.088		
CP-0.2	Context-Model	Sensitivity	0.494 $\pm$ 0.240	0.336 $\pm$ 0.139	0.467 $\pm$ 0.201	0.521 $\pm$ 0.131
		Specificity	0.760 $\pm$ 0.154	0.856 $\pm$ 0.057	0.747 $\pm$ 0.203	0.895 $\pm$ 0.046
		Accuracy	0.702 $\pm$ 0.076	0.675 $\pm$ 0.045	0.689 $\pm$ 0.157	0.820 $\pm$ 0.053
		False Positive Rate	0.240 $\pm$ 0.154	0.144 $\pm$ 0.057	0.253 $\pm$ 0.203	0.105 $\pm$ 0.046
		Overall Weighted Acc.		0.455 $\pm$ 0.078		
	Context-Label	Sensitivity	0.463 $\pm$ 0.231	0.262 $\pm$ 0.085	0.365 $\pm$ 0.205	0.452 $\pm$ 0.248
		Specificity	0.705 $\pm$ 0.173	0.838 $\pm$ 0.060	0.738 $\pm$ 0.194	0.865 $\pm$ 0.058
		Accuracy	0.660 $\pm$ 0.105	0.552 $\pm$ 0.021	0.678 $\pm$ 0.132	0.805 $\pm$ 0.062
		False Positive Rate	0.295 $\pm$ 0.173	0.162 $\pm$ 0.060	0.262 $\pm$ 0.194	0.136 $\pm$ 0.058
		Overall Weighted Acc.		0.386 $\pm$ 0.083		
CT-0.0	Context-Model	Sensitivity	0.545 $\pm$ 0.251	0.323 $\pm$ 0.128	0.412 $\pm$ 0.257	0.375 $\pm$ 0.090
		Specificity	0.679 $\pm$ 0.172	0.816 $\pm$ 0.090	0.796 $\pm$ 0.198	0.916 $\pm$ 0.074
		Accuracy	0.662 $\pm$ 0.082	0.636 $\pm$ 0.056	0.716 $\pm$ 0.138	0.817 $\pm$ 0.079
		False Positive Rate	0.322 $\pm$ 0.172	0.184 $\pm$ 0.090	0.204 $\pm$ 0.198	0.084 $\pm$ 0.074
		Overall Weighted Acc.		0.414 $\pm$ 0.078		
	Context-Label	Sensitivity	0.505 $\pm$ 0.248	0.276 $\pm$ 0.093	0.307 $\pm$ 0.238	0.286 $\pm$ 0.075
		Specificity	0.626 $\pm$ 0.186	0.809 $\pm$ 0.074	0.788 $\pm$ 0.192	0.886 $\pm$ 0.094
		Accuracy	0.605 $\pm$ 0.112	0.543 $\pm$ 0.015	0.711 $\pm$ 0.126	0.798 $\pm$ 0.086
		False Positive Rate	0.374 $\pm$ 0.186	0.191 $\pm$ 0.074	0.212 $\pm$ 0.192	0.114 $\pm$ 0.094
		Overall Weighted Acc.		0.343 $\pm$ 0.064		
CT-0.2	Context-Model	Sensitivity	0.534 $\pm$ 0.209	0.421 $\pm$ 0.088	0.400 $\pm$ 0.145	0.544 $\pm$ 0.207
		Specificity	0.724 $\pm$ 0.154	0.816 $\pm$ 0.046	0.865 $\pm$ 0.039	0.898 $\pm$ 0.052
		Accuracy	0.681 $\pm$ 0.076	0.687 $\pm$ 0.038	0.791 $\pm$ 0.029	0.841 $\pm$ 0.021
		False Positive Rate	0.276 $\pm$ 0.154	0.184 $\pm$ 0.046	0.135 $\pm$ 0.039	0.102 $\pm$ 0.052
		Overall Weighted Acc.		0.475 $\pm$ 0.076		
	Context-Label	Sensitivity	0.483 $\pm$ 0.191	0.322 $\pm$ 0.058	0.251 $\pm$ 0.129	0.496 $\pm$ 0.221
		Specificity	0.665 $\pm$ 0.163	0.785 $\pm$ 0.072	0.842 $\pm$ 0.016	0.857 $\pm$ 0.059
		Accuracy	0.630 $\pm$ 0.111	0.555 $\pm$ 0.029	0.747 $\pm$ 0.014	0.804 $\pm$ 0.056
		False Positive Rate	0.335 $\pm$ 0.163	0.215 $\pm$ 0.072	0.158 $\pm$ 0.016	0.144 $\pm$ 0.089
		Overall Weighted Acc.		0.388 $\pm$ 0.076		
DT-0.0	Context-Model	Sensitivity	0.489 $\pm$ 0.298	0.304 $\pm$ 0.182	0.243 $\pm$ 0.132	0.638 $\pm$ 0.267
		Specificity	0.660 $\pm$ 0.268	0.831 $\pm$ 0.061	0.893 $\pm$ 0.062	0.839 $\pm$ 0.092
		Accuracy	0.638 $\pm$ 0.128	0.665 $\pm$ 0.032	0.788 $\pm$ 0.038	0.815 $\pm$ 0.042
		False Positive Rate	0.340 $\pm$ 0.268	0.169 $\pm$ 0.061	0.108 $\pm$ 0.062	0.162 $\pm$ 0.092
		Overall Weighted Acc.		0.418 $\pm$ 0.075		
	Context-Label	Sensitivity	0.421 $\pm$ 0.297	0.236 $\pm$ 0.128	0.190 $\pm$ 0.122	0.585 $\pm$ 0.284
		Specificity	0.614 $\pm$ 0.283	0.794 $\pm$ 0.100	0.888 $\pm$ 0.054	0.793 $\pm$ 0.134
		Accuracy	0.578 $\pm$ 0.188	0.518 $\pm$ 0.036	0.776 $\pm$ 0.037	0.762 $\pm$ 0.096
		False Positive Rate	0.386 $\pm$ 0.283	0.206 $\pm$ 0.100	0.112 $\pm$ 0.054	0.207 $\pm$ 0.135
		Overall Weighted Acc.		0.358 $\pm$ 0.090		
DT-0.2	Context-Model	Sensitivity	0.640 $\pm$ 0.260	0.166 $\pm$ 0.058	0.261 $\pm$ 0.158	0.648 $\pm$ 0.300
		Specificity	0.563 $\pm$ 0.223	0.897 $\pm$ 0.070	0.912 $\pm$ 0.080	0.846 $\pm$ 0.088
		Accuracy	0.620 $\pm$ 0.117	0.650 $\pm$ 0.045	0.806 $\pm$ 0.059	0.823 $\pm$ 0.035
		False Positive Rate	0.438 $\pm$ 0.223	0.103 $\pm$ 0.070	0.088 $\pm$ 0.080	0.154 $\pm$ 0.088
		Overall Weighted Acc.		0.429 $\pm$ 0.092		
	Context-Label	Sensitivity	0.589 $\pm$ 0.252	0.138 $\pm$ 0.052	0.195 $\pm$ 0.107	0.602 $\pm$ 0.334
		Specificity	0.514 $\pm$ 0.233	0.880 $\pm$ 0.075	0.910 $\pm$ 0.080	0.801 $\pm$ 0.127
		Accuracy	0.530 $\pm$ 0.151	0.511 $\pm$ 0.020	0.794 $\pm$ 0.052	0.772 $\pm$ 0.087
		False Positive Rate	0.487 $\pm$ 0.233	0.120 $\pm$ 0.075	0.091 $\pm$ 0.080	0.199 $\pm$ 0.089
		Overall Weighted Acc.		0.381 $\pm$ 0.090		
SCNN	Model-Label	Sensitivity	0.707 $\pm$ 0.102	0.556 $\pm$ 0.068	0.325 $\pm$ 0.054	0.761 $\pm$ 0.069
		Specificity	0.778 $\pm$ 0.082	0.868 $\pm$ 0.084	0.887 $\pm$ 0.080	0.903 $\pm$ 0.062
		Accuracy	0.765 $\pm$ 0.058	0.714 $\pm$ 0.025	0.796 $\pm$ 0.072	0.882 $\pm$ 0.060
		False Positive Rate	0.222 $\pm$ 0.082	0.132 $\pm$ 0.084	0.113 $\pm$ 0.080	0.098 $\pm$ 0.062
		Overall Weighted Acc.		0.587 $\pm$ 0.037		

Table 9: Four-class classification results produced by four-fold cross-validation and context-based classification models.

Method	Comparison	Metric	Non-cancer	Cancer
CP-0.0	Context-Model	Sensitivity	0.911 $\pm$ 0.040	0.433 $\pm$ 0.187
		Specificity	0.433 $\pm$ 0.187	0.911 $\pm$ 0.040
		Accuracy	0.819 $\pm$ 0.041	0.819 $\pm$ 0.041
		False Positive Rate	0.567 $\pm$ 0.187	0.089 $\pm$ 0.040
		Overall Weighted Acc.	0.672 $\pm$ 0.076	
	Context-Label	Sensitivity	0.877 $\pm$ 0.061	0.345 $\pm$ 0.202
		Specificity	0.345 $\pm$ 0.202	0.877 $\pm$ 0.061
		Accuracy	0.800 $\pm$ 0.046	0.800 $\pm$ 0.046
		False Positive Rate	0.655 $\pm$ 0.202	0.123 $\pm$ 0.061
		Overall Weighted Acc.	0.611 $\pm$ 0.090	
CP-0.2	Context-Model	Sensitivity	0.895 $\pm$ 0.046	0.521 $\pm$ 0.131
		Specificity	0.521 $\pm$ 0.131	0.895 $\pm$ 0.046
		Accuracy	0.820 $\pm$ 0.053	0.820 $\pm$ 0.053
		False Positive Rate	0.479 $\pm$ 0.131	0.105 $\pm$ 0.046
		Overall Weighted Acc.	0.708 $\pm$ 0.055	
	Context-Label	Sensitivity	0.865 $\pm$ 0.058	0.452 $\pm$ 0.248
		Specificity	0.452 $\pm$ 0.248	0.865 $\pm$ 0.058
		Accuracy	0.804 $\pm$ 0.063	0.804 $\pm$ 0.063
		False Positive Rate	0.548 $\pm$ 0.248	0.136 $\pm$ 0.058
		Overall Weighted Acc.	0.658 $\pm$ 0.130	
CT-0.0	Context-Model	Sensitivity	0.916 $\pm$ 0.074	0.375 $\pm$ 0.090
		Specificity	0.375 $\pm$ 0.090	0.916 $\pm$ 0.074
		Accuracy	0.817 $\pm$ 0.079	0.817 $\pm$ 0.079
		False Positive Rate	0.626 $\pm$ 0.090	0.084 $\pm$ 0.074
		Overall Weighted Acc.	0.645 $\pm$ 0.053	
	Context-Label	Sensitivity	0.886 $\pm$ 0.094	0.286 $\pm$ 0.075
		Specificity	0.286 $\pm$ 0.075	0.886 $\pm$ 0.094
		Accuracy	0.798 $\pm$ 0.086	0.798 $\pm$ 0.086
		False Positive Rate	0.714 $\pm$ 0.075	0.114 $\pm$ 0.094
		Overall Weighted Acc.	0.586 $\pm$ 0.074	
CT-0.2	Context-Model	Sensitivity	0.898 $\pm$ 0.052	0.544 $\pm$ 0.207
		Specificity	0.544 $\pm$ 0.207	0.898 $\pm$ 0.052
		Accuracy	0.841 $\pm$ 0.021	0.841 $\pm$ 0.021
		False Positive Rate	0.456 $\pm$ 0.207	0.102 $\pm$ 0.052
		Overall Weighted Acc.	0.721 $\pm$ 0.077	
	Context-Label	Sensitivity	0.857 $\pm$ 0.089	0.496 $\pm$ 0.221
		Specificity	0.496 $\pm$ 0.221	0.857 $\pm$ 0.089
		Accuracy	0.804 $\pm$ 0.056	0.804 $\pm$ 0.056
		False Positive Rate	0.504 $\pm$ 0.221	0.144 $\pm$ 0.089
		Overall Weighted Acc.	0.676 $\pm$ 0.082	
DT-0.0	Context-Model	Sensitivity	0.839 $\pm$ 0.092	0.638 $\pm$ 0.267
		Specificity	0.638 $\pm$ 0.267	0.839 $\pm$ 0.092
		Accuracy	0.815 $\pm$ 0.042	0.815 $\pm$ 0.042
		False Positive Rate	0.362 $\pm$ 0.267	0.162 $\pm$ 0.092
		Overall Weighted Acc.	0.738 $\pm$ 0.095	
	Context-Label	Sensitivity	0.793 $\pm$ 0.134	0.585 $\pm$ 0.284
		Specificity	0.585 $\pm$ 0.284	0.793 $\pm$ 0.134
		Accuracy	0.762 $\pm$ 0.096	0.762 $\pm$ 0.096
		False Positive Rate	0.415 $\pm$ 0.284	0.207 $\pm$ 0.135
		Overall Weighted Acc.	0.689 $\pm$ 0.115	
DT-0.2	Context-Model	Sensitivity	0.847 $\pm$ 0.088	0.648 $\pm$ 0.300
		Specificity	0.648 $\pm$ 0.300	0.847 $\pm$ 0.088
		Accuracy	0.823 $\pm$ 0.035	0.823 $\pm$ 0.035
		False Positive Rate	0.352 $\pm$ 0.300	0.154 $\pm$ 0.088
		Overall Weighted Acc.	0.747 $\pm$ 0.109	
	Context-Label	Sensitivity	0.802 $\pm$ 0.127	0.602 $\pm$ 0.334
		Specificity	0.602 $\pm$ 0.334	0.802 $\pm$ 0.127
		Accuracy	0.772 $\pm$ 0.086	0.772 $\pm$ 0.086
		False Positive Rate	0.504 $\pm$ 0.334	0.144 $\pm$ 0.089
		Overall Weighted Acc.	0.677 $\pm$ 0.082	
SCNN	Model-Label	Sensitivity	0.903 $\pm$ 0.062	0.761 $\pm$ 0.069
		Specificity	0.761 $\pm$ 0.069	0.903 $\pm$ 0.062
		Accuracy	0.882 $\pm$ 0.060	0.882 $\pm$ 0.060
		False Positive Rate	0.239 $\pm$ 0.069	0.098 $\pm$ 0.062
		Overall Weighted Acc.	0.832 $\pm$ 0.059	

Table 10: Binary classification results produced by four-fold cross-validation and context-based classification models.